SIP API

for Java™ 2 Micro Edition

Final Release
Version 1.0

JSR 180 Expert Group
jsr-180-comments@jcp.org

Java Community Process (JCP)

JSR-180 SIP API for J2ME Specification ("Specification")

Version: 1.0

Status: Final Release

Specification Lead: Nokia Corporation ("Specification Lead")

Release: 2003-11-18

NOTICE; LIMITED LICENSE GRANTS

Specification Lead hereby grants You a fully-paid, non-exclusive, non-transferable, worldwide, limited license (without the right to sublicense), under the Specification Lead's applicable intellectual property rights to view, download, use and reproduce the Specification only for the purpose of internal evaluation, which shall be understood to include developing applications intended to run on an implementation of the Specification provided that such applications do not themselves implement any portion(s) of the Specification. The Specification contains proprietary information of the Specification Lead and may only be used in accordance with the license terms set forth herein.

Subject to the reciprocity requirement set forth below Specification Lead also grants You a perpetual, non-exclusive, worldwide, fully paid-up, royalty free, irrevocable limited license (without the right to sublicense) under any applicable copyrights or patent rights it may have in the Specification to create and/or distribute an Independent Implementation of the Specification that: (a) fully implements the Specification without modifying, subsetting or extending the public class or interface declarations whose names begin with "java" or "javax" or their equivalents in any subsequent naming convention adopted by Specification Lead through the Java Community Process, or any recognized successors or replacements thereof; (b) implement all required interfaces and functionality of the Specification; (c) only include as part of such Independent Implementation the packages, classes or methods specified by the Specification; (d) pass the technology compatibility kit ("TCK") for such Specification; and (e) are designed to operate on a Java platform which is certified to pass the complete TCK for such Java platform. For the purpose of this agreement the applicable patent rights shall mean any claims for which there is no technically feasible way of avoiding infringement in the course of implementing the Specification. Other than this limited license, You acquire no right, license, title or interest in or to the Specification or any other intellectual property rights of the Specification Lead.

You need not include limitations (a)-(e) from the previous paragraph or any other particular "pass through" requirements in any license You grant concerning the use of Your Independent Implementation or products derived from it. However, except with respect to implementations of the Specification (and products derived from them) by Your licensee that satisfy limitations (a)-(e) from the previous paragraph, You may neither: (i) grant or otherwise pass through to Your licensees any licenses under Specification Lead's applicable intellectual property rights; nor (ii) authorize Your licensees to make any claims concerning their implementation's compliance with the Specification in question.

The license provisions concerning the grant of licenses hereunder shall be subject to reciprocity requirement so that Specification Lead's grant of licenses shall not be effective as to You if, with respect to the Specification licensed hereunder, You (on behalf of yourself and any party for which You are authorized to act with respect to this Agreement) do not make available, in fact and practice, to Specification Lead and to other licensees of Specification on fair, reasonable and non-discriminatory terms a perpetual, irrevocable, non-exclusive, non-transferable, worldwide license under such Your (and such party's for which You are authorized to act with respect to this Agreement) patent rights which are or would be infringed by all technically feasible implementations of the Specification to develop, distribute and use an Independent Implementation of the Specification within the scope of the licenses granted above by Specification Lead. However, You shall not be required to grant a license:

1.to a licensee not willing to grant a reciprocal license under its patent rights to You and to any other party seeking such a license with respect to the enforcement of such licensee's patent claims where there is no technically feasible alternative that would avoid the infringement of such claims;

2.with respect to any portion of any product and any combinations thereof the sole purpose or function of which is not required in order to be fully compliant with the Specification; or

3.with respect to technology that is not required for developing, distributing and using an Independent Implementation.

Furthermore, You hereby grant a non-exclusive, worldwide, royalty-free, perpetual and irrevocable covenant to Specification Lead that You shall not bring a suit before any court or administrative agency or otherwise assert a claim that the Specification Lead has, in the course of performing its responsibilities as the Specification Lead under JCP process rules, induced any other entity to infringe Your patent rights.

For the purposes of this Agreement: "Independent Implementation" shall mean an implementation of the Specification that neither derives from the reference implementation to the Specification ("Reference Implementation") source code or binary code materials nor, except with an appropriate and separate license from licensor of the Reference Implementation, includes any of Reference Implementation's source code or binary code materials.

This Agreement will terminate immediately without notice from Specification Lead if You fail to comply with any material provision of or act outside the scope of the licenses granted above.

TRADEMARKS

Nokia is a registered trademark of Nokia Corporation. Nokia Corporation 's product names are either trademarks or registered trademarks of Nokia Corporation Your access to this Specification should not be construed as granting, by implication, estoppel or otherwise, any license or right to use any marks appearing in the Specification without the prior written consent of Nokia Corporation or Nokia's licensors .

No right, title, or interest in or to any trademarks, service marks, or trade names of Sun or Sun's licensors, is granted hereunder. Sun, Sun Microsystems, the Sun logo, Java, J2ME, and the Java Coffee Cup logo are trademarks or registered trademarks of Sun Microsystems, Inc. in the U.S. and other countries.

DISCLAIMER OF WARRANTIES

THE SPECIFICATION IS PROVIDED "AS IS". SPECIFICATION LEAD MAKES NO REPRESENTATIONS OR WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT, THAT THE CONTENTS OF THE SPECIFICATION ARE SUITABLE FOR ANY PURPOSE OR THAT ANY PRACTICE OR IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADE SECRETS OR OTHER RIGHTS. This document does not represent any commitment to release or implement any portion of the Specification in any product.

THE SPECIFICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION THEREIN; THESE CHANGES WILL BE INCORPORATED INTO NEW VERSIONS OF THE SPECIFICATION, IF ANY. SPECIFICATION LEAD MAY MAKE IMPROVEMENTS AND/OR CHANGES TO THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THE SPECIFICATION AT ANY TIME. Any use of such changes in the Specification will be governed by the then-current license for the applicable version of the Specification.

LIMITATION OF LIABILITY

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL SPECIFICATION LEAD OR ITS LICENSORS BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION, LOST REVENUE, PROFITS OR DATA, OR FOR SPECIAL, INDIRECT, CONSEQUENTIAL, INCIDENTAL OR PUNITIVE DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF OR RELATED TO ANY FURNISHING, PRACTICING, MODIFYING OR ANY USE OF THE SPECIFICATION, EVEN IF SPECIFICATION LEAD AND/OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

You will indemnify, hold harmless, and defend Specification Lead and its licensors from any claims arising or resulting from: (i) Your use of the Specification; (ii) the use or distribution of Your Java application, applet and/or clean room implementation; and/or (iii) any claims that later versions or releases of any Specification furnished to You are incompatible with the Specification provided to You under this license.

RESTRICTED RIGHTS LEGEND

U.S. Government: If this Specification is being acquired by or on behalf of the U.S. Government or by a U.S. Government prime contractor or subcontractor (at any tier), then the Government's rights in the Software and accompanying documentation shall be only as set forth in this license; this is in accordance with 48 C.F.R. 227.7201 through 227.7202-4 (for Department of Defense (DoD) acquisitions) and with 48 C.F.R. 2.101 and 12.212 (for non-DoD acquisitions).

REPORT

You may wish to report any ambiguities, inconsistencies or inaccuracies You may find in connection with Your use of the Specification ("Feedback"). To the extent that You provide Specification Lead with any Feedback, You hereby: (i) agree that such Feedback is provided on a non-proprietary and non-confidential basis, and (ii) grant Specification Lead a perpetual, non-exclusive, worldwide, fully paid-up, irrevocable license, with the right to sublicense through multiple levels of sublicensees, to incorporate, disclose, and use without limitation the Feedback for any purpose related to the Specification and future versions, implementations, and test suites thereof.

Contents

CHAPTER 1

Overview

Description

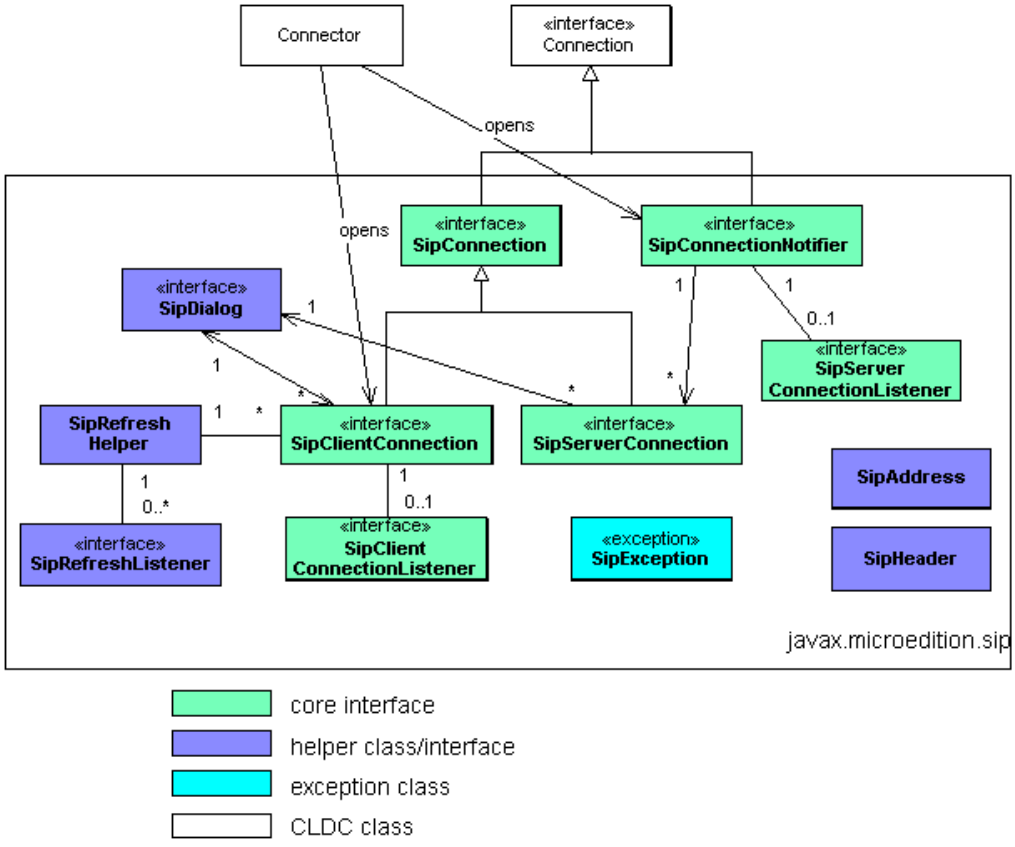
This specification defines the JSR180, SIP API for J2ME. This javadoc documentation contains Proposed Final Draft according to the Java Community Process. This document and all associated documents are subject to the terms of the JCP (2.1) agreements (i.e. JSPA and/or IEPA).

Overview

This specification defines a J2ME Optional Package that enables resource limited devices (referred to as 'terminals' in the following) to send and receive SIP messages. The API is designed to be a compact and generic SIP API, which provides SIP functionality in transaction level. The API is integrated into the Generic Connection Framework defined in Connected, Limited Device Configuration (CLDC).

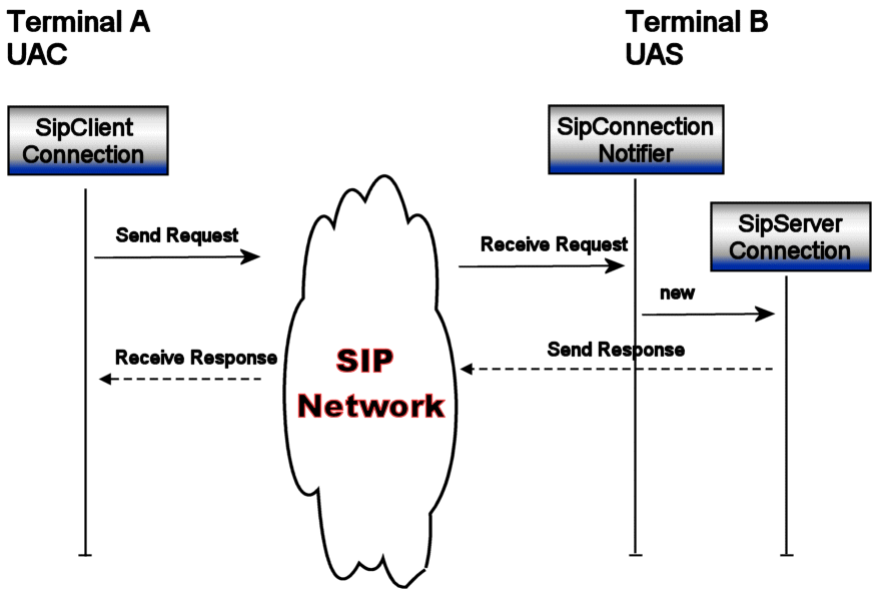
The SIP API for J2ME is designed as an Optional Package that can be used with many J2ME Profiles. The minimum platform required by this API is the J2ME CLDC v1.0. The API can also be used with the J2ME Connected Device Configuration (CDC).

The picture below shows the simplified class diagram of the API, relations between classes, inheritance from `javax.microedition.Connection` and relation to `javax.microedition.Connector`.

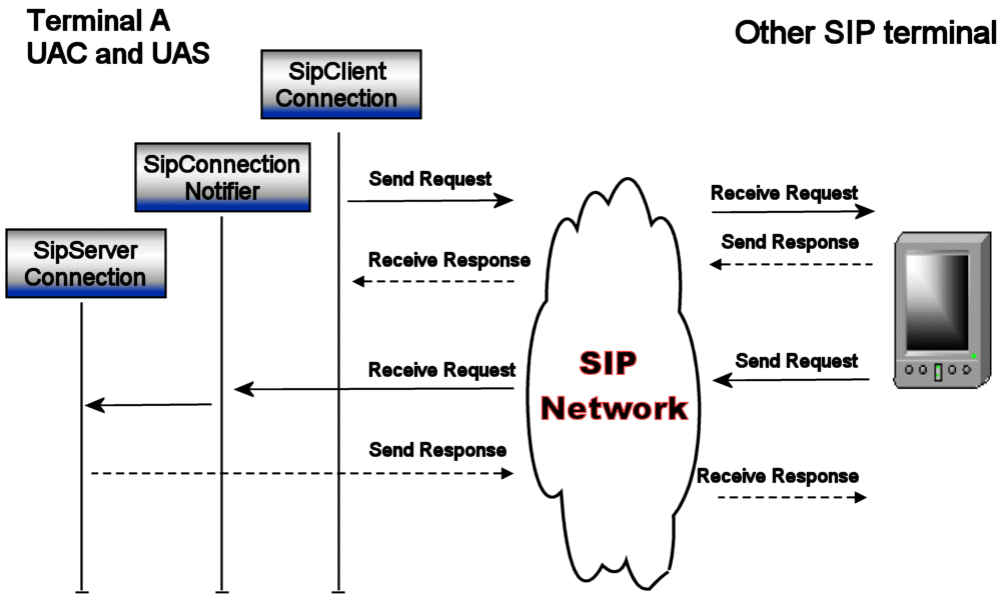


How to use the SIP API for J2ME

The SIP API is used by the applications to implement SIP User Agent (UA) functionality. The following picture shows what interfaces terminals A and B are using to implement SIP User Agent Client (UAC) and User Agent Server (UAS) functionality respectively. See Glossary for explanation of UAC and UAS functionality.



In reality applications will use both SIP client and server connections in the same terminal (terminal A in picture below) and thus implementing both UAC and UAS functionality.



The following code examples shows how to open SIP connections and the usage of the main classes `SipClientConnection`, `SipServerConnection` and `SipConnectionNotifier`. Examples of helper classes: `SipDialog`, `SipAddress`, `SipHeader` and `RefreshHelper` are shown in the API

documentation.

The example below shows how to open a SIP client connection, send one request and receive a response. The interface used in this simple client connection example is `SipClientConnection`.

```
public void sendTextMessage(String msg) {
   SipClientConnection sc = null;
   try {
      // open SIP connection
      sc = (SipClientConnection) Connector.open("sip:sippy.tester@host.com:5060");

      // initialize SIP request MESSAGE
      sc.initRequest("MESSAGE", null);

      // set one header
      sc.setHeader("Subject", "testing...");

      // write message body
      sc.setHeader("Content-Type", "text/plain");
      sc.setHeader("Content-Length", Integer.toString(msg.length()));
      OutputStream os = sc.openContentOutputStream();
      os.write(msg.getBytes());
      os.close(); // close stream and send the message to the network

      // wait max 15 seconds for response
      sc.receive(15000);

      // response received
      if(sc.getStatusCode() == 200) {
         // handle 200 OK response
      } else {
         // handle other responses
      }
      sc.close();
    } catch(Exception ex)
      // handle Exceptions
    }
}
```

The following code example shows how to open a SIP server connection, receive one request and send a response. The interfaces used in the simple server connection example are `SipConnectionNotifier` and `SipServerConnection`.

```
public receiveMessage() {
   SipConnectionNotifier scn = null;
   SipServerConnection ssc = null;
   String method = null;

   try {
      // Open SIP server connection and listen to incoming requests
      scn = (SipConnectionNotifier) Connector.open("sip:");

      // block and wait for incoming request.
      // SipServerConnection is established and returned
      // when a new request is received.
      ssc = scn.acceptAndOpen();

      // what was the SIP method
      method = ssc.getMethod();
      if(method.equals("MESSAGE")) {
         // read the content of the MESSAGE
         String contentType = ssc.getHeader("Content-Type");
         if((contentType != null) && contentType.equals("text/plain")) {
            InputStream is = ssc.openContentInputStream();
            int ch;
            // read content
            while ((ch = is.read()) != -1) {
               ...
            }
         }
         // initialize SIP 200 OK and send it back
         ssc.initResponse(200);
         ssc.send();
      }
      ssc.close();
   } catch(Exception ex) {
     // handle IOException, InterruptedIOException, SecurityException
     // or SipException
   }
}
```

SIP (Session Initiation Protocol) Overview

SIP is an application-layer control protocol that can establish, modify, and terminate multimedia sessions. SIP can also invite participants to already existing sessions. SIP transparently supports name mapping and redirection services, which supports personal mobility - users can maintain a single externally visible identifier

regardless of their network location (see RFC 3261 [1]). SIP can also be used to implement non-real-time services like Instant Messaging and Presence [2][3].

SIP supports five facets of establishing and terminating multimedia communications:

User location

- determination of the end system to be used for communication

User availability

- determination of the willingness of the called party to engage in communications

User capabilities

- determination of the media and media parameters to be used

Session setup

- establishment of session parameters at both called and calling party

Session management

- including transfer and termination of sessions, modifying session parameters, and invoking services

Glossary

This section defines some terms used in the specification. Please also refer to the RFC 3261 [1].

Client Connection	The SIP API's client connection (class `SipClientConnection`) is an interface to SIP client transaction, which sends original request and receives responses until final response. See also, *SIP Transaction* and *Server Connection*.
Server Connection	The SIP API's server connection (class `SipServerConnection`) is an interface to SIP server transaction, which receives the original request and sends response(s) until final response. Also see, *SIP Transaction* and *Client Connection*.
SIP Transaction	A SIP transaction occurs between a client and a server and comprises all messages from the first request sent from the client to the server up to a final (non-1xx) response sent from the server to the client. If the request is INVITE and the final response is a non-2xx, the transaction also includes an ACK to the response. The ACK for a 2xx response to an INVITE request is a separate transaction.
SIP Dialog	The SIP API's class `SipDialog` keeps the information about one SIP dialog. Using that interface the user is able to send subsequent requests within that dialog. A dialog is a peer-to-peer SIP relationship between two UAs that persists for some time. A dialog is established by SIP messages, such as a 2xx response to an INVITE request. A dialog is identified by a call identifier, local tag, and a remote tag.

User Agent (UA)	A logical entity that can act as both a user agent client and user agent server. Typically an application (e.g. MIDlet) will implement the User Agent functionality using the SIP API for J2ME. See also, *UAC* and *UAS*.
User Agent Client (UAC)	A user agent client is a logical entity that creates a new request, and then uses the client transaction state machinery to send it. The role of the UAC lasts only for the duration of that transaction. In other words, if a piece of software initiates a request, it acts as a UAC for the duration of that transaction. If it receives a request later, it assumes the role of a user agent server for the processing of that transaction.
User Agent Server (UAS)	A user agent server is a logical entity that generates a response to a SIP request. The response accepts, rejects, or redirects the request. This role lasts only for the duration of that transaction. In other words, if a piece of software responds to a request, it acts as a UAS for the duration of that transaction. If it generates a request later, it assumes the role of a user agent client for the processing of that transaction.
Method	The method is the primary function that a request is meant to invoke on a server. The method is carried in the request message itself. Example methods are INVITE and BYE.
Provisional Response	A response used by the server to indicate progress, but that does not terminate a SIP transaction. 1xx responses are provisional, other responses are considered final.
Final Response	A response that terminates a SIP transaction, as opposed to a provisional response that does not. All 2xx, 3xx, 4xx, 5xx and 6xx responses are final.

Notation used

This specification uses the following terms as they are defined in the table below:

Term	Definition
MUST	The associated definition is an absolute requirement of this specification.
MUST NOT	The definition is an absolute prohibition of this specification.
SHOULD	Indicates a recommended practice. There may exist valid reasons in particular circumstances to ignore this recommendation, but the full implications must be understood and carefully weighed before choosing a different course.

SHOULD NOT	Indicates a non-recommended practice. There may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.
MAY	Indicates that an item is truly optional.

References

[1]	SIP: Session Initiation Protocol, RFC 3261, June 2002
[2]	Session Initiation Protocol (SIP)-Specific Event Notification, RFC 3265, June 2002
[3]	Session Initiation Protocol (SIP) Extension for Instant Messaging, RFC 3428, December 2002

Package Summary

Packages

javax.microedition.sip 7	This specification defines the JSR180, SIP API for J2ME.

Class Hierarchy

```
java.lang.Object
    javax.microedition.sip.SipAddress 42
    javax.microedition.sip.SipHeader 37
    javax.microedition.sip.SipRefreshHelper 48
    java.lang.Throwable
        java.lang.Exception
            java.io.IOException
                javax.microedition.sip.SipException 53
```

Interface Hierarchy

```
javax.microedition.io.Connection
    javax.microedition.sip.SipConnection 9
        javax.microedition.sip.SipClientConnection 16
        javax.microedition.sip.SipServerConnection 24
    javax.microedition.sip.SipConnectionNotifier 28
javax.microedition.sip.SipClientConnectionListener 31
javax.microedition.sip.SipDialog 33
javax.microedition.sip.SipRefreshListener 52
javax.microedition.sip.SipServerConnectionListener 32
```

CHAPTER 2

Package javax.microedition.sip

Description

This specification defines the JSR180, SIP API for J2ME.

This document and all associated documents are subject to the terms of the JCP (2.1) agreements (i.e. JSPA and/or IEPA).

References

[1]	SIP: Session Initiation Protocol, RFC 3261, June 2002
[2]	Session Initiation Protocol (SIP)-Specific Event Notification, RFC 3265, June 2002
[3]	Session Initiation Protocol (SIP) Extension for Instant Messaging, RFC 3428, December 2002

For additional information please refer to following appendixes:

- Annex A: Deploying JSR 180 on MIDP 2.0 Platform
- Annex B: Code example MIDlets

Class Summary

Interfaces

`SipClientConnection` 16	`SipClientConnection` represents SIP client transaction.
`SipClientConnectionListener` 31	Listener interface for incoming SIP responses.
`SipConnection` 9	`SipConnection` is the base interface for SIP connections.
`SipConnectionNotifier` 28	This interface defines a SIP server connection notifier.
`SipDialog` 33	`SipDialog` represents one SIP Dialog.
`SipRefreshListener` 52	Listener interface for `RefreshHelper` events.
`SipServerConnection` 24	`SipServerConnection` represents SIP server transaction.
`SipServerConnectionListener` 32	Listener interface for incoming SIP requests.

Classes

Class Summary	
SipAddress 42	SipAddress provides a generic SIP address parser.
SipHeader 37	SipHeader provides generic SIP header parser helper.
SipRefreshHelper 48	This class implements the functionality that facilitates the handling of refreshing requests on behalf of the application.
Exceptions	
SipException 53	This is an exception class for SIP specific errors.

javax.microedition.sip
SipConnection

Declaration
```
public interface SipConnection extends javax.microedition.io.Connection
```

All Superinterfaces: `javax.microedition.io.Connection`

All Known Subinterfaces: `SipClientConnection`[16], `SipServerConnection`[24]

Description
`SipConnection` is the base interface for SIP connections. `SipConnection` holds the common properties and methods for subinterfaces `SipClientConnection` and `SipServerConnection`.

Integration to Generic Connection Framework

Integration to `javax.microedition.io.Connector`
A new SIP connection is instantiated by a call to `Connector.open()`. An application SHOULD call `close()` when it is finished with the connection. It should be noticed that the `Connector.open()` returns a *client mode connection* (`SipClientConnection`) or *server mode connection* (`SipConnectionNotifier`) depending on the string (SIP URI) passed to the `Connector.open()`. The SIP URI is specified in RFC3261 [1] and here it takes general form of:

```
{scheme}:[{target}][{params}]
```

where:

- **`scheme`** is SIP scheme supported by the system `sip` or `sips`
- **`target`** is user network address in form of `{user_name}@{target_host}[:{port}]` or `{telephone_number}`, see examples below.
- **`params`** stands for additional SIP URI parameters like `;transport=udp`

Opening new client connection (`SipClientConnection`)
If the target host is included a, `SipClientConnection` will be returned.
Examples:

```
sip:bob@biloxi.com
sip:alice@10.128.0.8:5060
sips:alice@company.com:5060;transport=tcp
sip:+358-555-1234567;postd=pp22@foo.com;user=phone
```

Opening new server connection (`SipConnectionNotifier`)
Three forms of SIP URIs are allowed where the target host is omitted:

- **`sip:nnnn`**, returns `SipConnectionNotifier` listening to incoming SIP requests on port number `nnnn`.

- **`sip:`**, returns `SipConnectionNotifier` listening to incoming SIP requests on a port number allocated by the system.
- **`sip:[nnnn];type="application/x-type"`**, returns `SipConnectionNotifier` listening to incoming SIP requests that match to the MIME type "application/x-type" (specified in the URI parameter `type`). Port number is optional. See SIP Identity for more information about this option.

Examples of opening a SIP connection:

```
SipClientConnection scn = (SipClientConnection) Connector.open("sip:bob@biloxi.com");
SipClientConnection scn =
  (SipClientConnection) Connector.open("sips:alice@company.com:5060;transport=tcp");
SipConnectionNotifier scn = (SipConnectionNotifier) Connector.open("sip:5060");
SipConnectionNotifier scn = (SipConnectionNotifier) Connector.open("sips:5080");
SipConnectionNotifier scn = (SipConnectionNotifier) Connector.open("sip:");
```

The optional second parameter in `Connector.open()` specifies the access mode. This is ignored in SIP API. This is because both connection modes: *client mode connection* and *server mode connection* can send and receive messages.

The optional third parameter is a boolean flag that indicates if the calling code can handle timeout exceptions. That parameter is also ignored by the SIP API.

The `Connector` convenience methods to gain access to a specific input or output stream directly are not supported by the SIP API. The implementations MUST throw `IOException` if these methods are called with SIP URIs.

The `Connector.open()` method throws following Exceptions:

`IllegalArgumentException`

- If a parameter is invalid.

`ConnectionNotFoundException`

- If the requested connection cannot be created, or the protocol type does not exist.

`IOException`

- If some other kind of I/O error occurs. For example the port number for *server mode connection* is already reserved.

`SecurityException`

- May be thrown if access to the protocol handler is prohibited.

`SipException`

- TRANSACTION_UNAVAILABLE if the system can not open new SIP transactions. TRANSPORT_NOT_SUPPORTED if the requested transport is not supported. See above `transport` URI parameter.

Identification of the SIP API

To enable applications to test for the presence of the SIP API and its version during runtime, a system property is defined. Platforms where this API is implemented according to this specification shall include a system property with a key **`"microedition.sip.version"`**. When `System.getProperty` is called with this key, implementations conforming to this specification shall return the version string **`"1.0"`**.

SIP Identity

SIP identity is a logical identity (address-of-record), a type of Uniform Resource Identifier (URI) called a SIP URI. SIP identity is frequently thought of as the "public address" of the user. (see RFC 3261 [1], 6 Definitions, p.20)

A terminal that supports SIP is typically configured for one user at the time (SIP provider settings). This means that the SIP system uses that address-of-record (URI) as the identity when communicating to other entities. Now, it is very likely that there will be several applications that wants to share the same SIP identity for example: audio/video, instant message/presence and e.g. game applications. On the other hand, some applications can use their own SIP identity (for that particular service), which does not share the system SIP properties.

The issue falls to the decisions on how the SIP requests are routed to the User Agents (applications). This functionality is specified in SIP Working Group (http://www.ietf.org/html.charters/sip-charter.html) as callee's capabilities and caller's preferences. The implementations must follow the framework and parameters specified in the final specification of the SIP working group.

When sharing the same identity (address-of-record) the terminal has to know where the incoming request is routed. To do this it can use the MIME type tag carried in a special header (e.g. `Accept-Contact`). Each application is mapped to a certain MIME type. Java SIP API provides the way to utilize this functionality as is specified below.

To allow the Java SIP API to share the system SIP identity or use application's own identity, the following rules are defined when the SIP server connection is opened with `Connector.open()`:

	Connector SIP URI	SIP Identity	Request routing
1)	`sip:5080[;type= "application/x-game"]`	- use dedicated port number `5080` - use identity set by the application	- route all incoming SIP requests on port 5080 to this `SipConnectionNotifier`. If the optional MIME type feature tag `"application/x-game"` is set, route only the requests with the matching tag.
2)	`sip:;type= "application/x-game"`	- share system SIP port - share system SIP identity	- route incoming SIP requests with MIME type feature tag `"application/x-game"` to this `SipConnectionNotifier`
3)	`sip:`	- use dedicated port selected by the system - use identity set by the application	- route all incoming SIP requests to the selected port to this `SipConnectionNotifier`

See Also: `SipClientConnection`[16], `SipServerConnection`[24], `SipConnectionNotifier`[28]

Member Summary

Methods

`void`	`addHeader(java.lang.String name, java.lang.String value)`[13]
`SipDialog`	`getDialog()`[15]
`java.lang.String`	`getHeader(java.lang.String name)`[14]

Member Summary	
java.lang.String[]	getHeaders(java.lang.String name)[13]
java.lang.String	getMethod()[14]
java.lang.String	getReasonPhrase()[14]
java.lang.String	getRequestURI()[14]
int	getStatusCode()[14]
java.io.InputStream	openContentInputStream()[15]
java.io.OutputStream	openContentOutputStream()[15]
void	removeHeader(java.lang.String name)[13]
void	send()[12]
void	setHeader(java.lang.String name, java.lang.String value)[12]

Inherited Member Summary
Methods inherited from interface `Connection`
`close()`

Methods

send()

Declaration:

```
public void send()
            throws IOException, InterruptedIOException, SipException
```

Description:

Sends the SIP message. Send must also close the `OutputStream` if it was opened.

Throws:

`java.io.IOException` - if the message could not be sent or because of network failure.

`java.io.InterruptedIOException` - if a timeout occurs while either trying to send the message or if this `Connection` object is closed during this `send` operation

`SipException`[53] - INVALID_STATE if the message can not be sent in this state. INVALID_MESSAGE there was an error in message format.

setHeader(String, String)

Declaration:

```
public void setHeader(java.lang.String name, java.lang.String value)
            throws SipException, IllegalArgumentException
```

Description:

Sets header value in SIP message. If the header does not exist it will be added to the message, otherwise the existing header is overwritten. If multiple header field values exist the topmost is overwritten. The implementations MAY restrict the access to some headers according to RFC 3261 [1].

Parameters:

`name` - name of the header, either in full or compact form see [1] p.32

`value` - the header value

Throws:

`java.lang.IllegalArgumentException` - MAY be thrown if the header or value is invalid.

`SipException`[53] - INVALID_STATE if header can not be set in this state. INVALID_OPERATION if the system does not allow to set this header.

addHeader(String, String)

Declaration:

```
public void addHeader(java.lang.String name, java.lang.String value)
            throws SipException, IllegalArgumentException
```

Description:

Adds a header to the SIP message. If multiple header field values exist the header value is added topmost of this type of headers. The implementations MAY restrict the access to some headers according to RFC 3261 [1].

Parameters:

`name` - name of the header, either in full or compact form see [1] p.32

`value` - the header value

Throws:

`java.lang.IllegalArgumentException` - MAY be thrown if the header or value is invalid.

`SipException`[53] - INVALID_STATE if header can not be added in this state. INVALID_OPERATION if the system does not allow to add this header.

removeHeader(String)

Declaration:

```
public void removeHeader(java.lang.String name)
            throws SipException
```

Description:

Removes header from the message. If multiple header field values exist the topmost is removed. The implementations MAY restrict the access to some headers according to RFC 3261 [1]. If the named header is not found this method does nothing.

Parameters:

`name` - name of the header to be removed, either in full or compact form see [1] p.32

Throws:

`SipException`[53] - INVALID_STATE if header can not be removed in this state. INVALID_OPERATION if the system does not allow to remove this header.

getHeaders(String)

Declaration:

```
public java.lang.String[] getHeaders(java.lang.String name)
```

Description:

Gets the header field value(s) of specified header type.

Parameters:

`name` - name of the header type, either in full or compact form see [1] p.32

Returns: array of header field values (topmost first), or `null` if the current message does not have such a header or the header is for other reason not available (e.g. message not initialized).

getHeader(String)

Declaration:
```
public java.lang.String getHeader(java.lang.String name)
```

Description:
Gets the header field value of specified header type.

Parameters:
`name` - name of the header type, either in full or compact form see [1] p.32

Returns: topmost header field value, or `null` if the current message does not have such a header or the header is for other reason not available (e.g. message not initialized).

getMethod()

Declaration:
```
public java.lang.String getMethod()
```

Description:
Gets the SIP method. Applicable when a message has been initialized or received.

Returns: SIP method name REGISTER, INVITE, NOTIFY, etc. Returns `null` if the method is not available.

getRequestURI()

Declaration:
```
public java.lang.String getRequestURI()
```

Description:
Gets Request-URI. Available when `SipClientConnection` is in *Initialized* state or when `SipServerConnection` is in *Request Received* state. Built from the original URI given in `Connector.open()`. See RFC 3261 p.35 (8.1.1.1 Request-URI)

Returns: Request-URI of the message. Returns `null` if the Request-URI is not available.

getStatusCode()

Declaration:
```
public int getStatusCode()
```

Description:
Gets SIP response status code. Available when `SipClientConnection` is in *Proceeding* or *Completed* state or when `SipServerConnection` is in *Initialized* state.

Returns: status code 1xx, 2xx, 3xx, 4xx, ... Returns 0 if the status code is not available.

getReasonPhrase()

Declaration:
```
public java.lang.String getReasonPhrase()
```

Description:
Gets SIP response reason phrase. Available when `SipClientConnection` is in *Proceeding* or *Completed* state or when `SipServerConnection` is in *Initialized* state.

Returns: reason phrase. Returns `null` if the reason phrase is not available.

getDialog()

Declaration:
```
public javax.microedition.sip.SipDialog33 getDialog()
```

Description:
Returns the current SIP dialog. This is available when the `SipConnection` belongs to a created `SipDialog` and the system has received (or sent) provisional (101-199) or final response (200).

Returns: `SipDialog` object if this connection belongs to a dialog, otherwise returns `null`.

See Also: `SipDialog`[33]

openContentInputStream()

Declaration:
```
public java.io.InputStream openContentInputStream()
            throws IOException, SipException
```

Description:
Returns `InputStream` to read SIP message body content.

Returns: InputStream to read body content

Throws:

`java.io.IOException` - if the `InputStream` can not be opened, because of an I/O error occurred.

`SipException`[53] - INVALID_STATE the `InputStream` can not be opened in this state (e.g. no message received).

openContentOutputStream()

Declaration:
```
public java.io.OutputStream openContentOutputStream()
            throws IOException, SipException
```

Description:
Returns `OutputStream` to fill the SIP message body content. When calling `close()` on `OutputStream` the message will be sent to the network. So it is equivalent to call `send()`. Again `send()` must not be called after closing the `OutputStream`, since it will throw Exception because of calling the method in wrong state.
Before opening `OutputStream` the `Content-Length` and `Content-Type` headers has to se set. If not `SipException.UNKNOWN_LENGTH` or `SipException.UNKNOWN_TYPE` will be thrown respectively.

Returns: OutputStream to write body content

Throws:

`java.io.IOException` - if the `OutputStream` can not be opened, because of an I/O error occurred.

`SipException`[53] - INVALID_STATE the `OutputStream` can not be opened in this state (e.g. no message initialized). UNKNOWN_LENGTH `Content-Length` header not set. UNKNOWN_TYPE `Content-Type` header not set.

See Also: `send()`[12]

javax.microedition.sip
SipClientConnection

Declaration

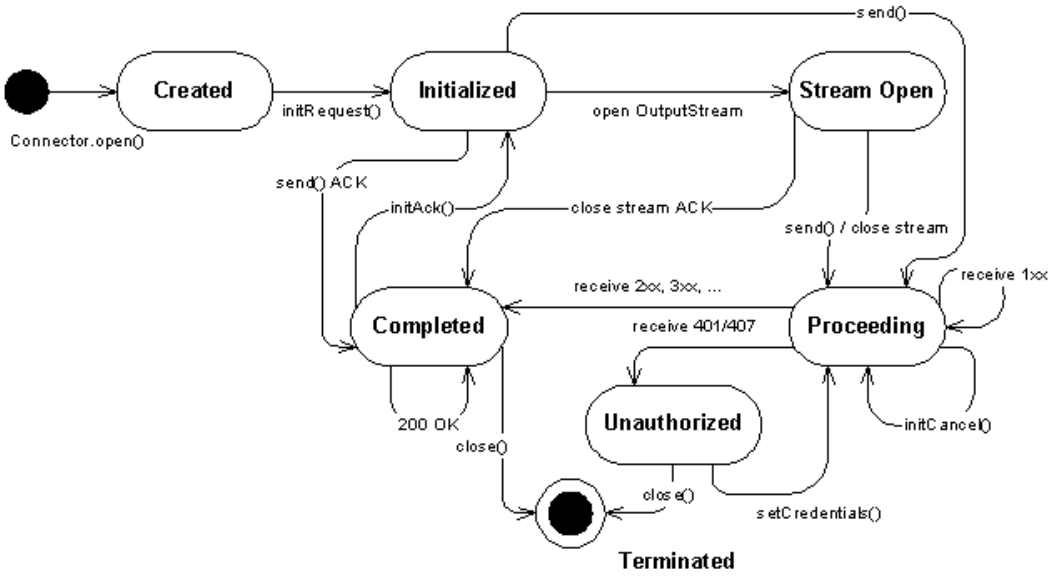
```
public interface SipClientConnection extends SipConnection9
```

All Superinterfaces: `javax.microedition.io.Connection`, `SipConnection9`

Description

`SipClientConnection` represents SIP client transaction. application can create a new `SipClientConnection` with `Connector` or `SipDialog` object.

The `SipClientConnection` has following state chart:



- *Created*, `SipClientConnection` created from `Connector` or `SipDialog`
- *Initialized*, request has been initialized with `initRequest(...)` or `initAck()` or `initCancel()`
- *Stream Open*, `OutputStream` opened with `openContentOutputStream()`. Opening `InputStream` for received response does not trigger state transition.
- *Proceeding*, request has been sent, waiting for the response, or provisional 1xx response received. `initCancel()` can be called, which will spawn a new `SipClientConnection` which is in *Initialized* state
- *Completed*, transaction completed with final response (2xx, 3xx, 4xx, 5xx, 6xx) in this state the ACK can be initialized. Multiple 200 OK responses can be received. Note different state transition for responses 401 and 407.
- *Unauthorized*, transaction completed with response 401 (Unauthorized) or 407 (Proxy Authentication Required). The application can re-originate the request with proper credentials by calling `setCredentials()` method. After this the `SipClientConnection` is back in *Proceeding* state.

- *Terminated*, the final state, in which the SIP connection has been terminated by error or closed

Note: The state chart of SipClientConnection differs from the state chart of SIP client transaction, which can be found in RFC 3261 [1] p.128-133

Following methods are restricted to a certain state. The table shows the list of restricted methods allowed in each state.

- *Created*
  `initRequest`

- *Initialized*
  `addHeader`
  `setHeader`
  `removeHeader`
  `setRequestURI`
  `openContentOutputStream`
  `send`
  `enableRefresh`
  `setCredentials`

- *Stream Open*
  Methods in `OutputStream` and `SipConnection.send`
- *Proceeding*
  `receive`
  `openContentInputStream`
  `initCancel`

- *Completed*
  `openContentInputStream`
  `initAck`

- *Unauthorized*
  `setCredentials`

- *Terminated*
  The transaction and this connection is closed. The I/O methods above will throw IOException

Following methods can be called in every state. The functionality is defined by the method depending of the information availability.

- *Can be called in every state*
  `getHeader`
  `getHeaders`
  `getRequestURI`
  `getMethod`
  `getStatusCode`
  `getReasonPhrase`

`getDialog`
`setListener`
`close` // causes state transition to *Terminated* state

Code examples

The example below illustrates the usage of SIP client connection: opening, sending one request (MESSAGE) and receiving response:

```
public void sendTextMessage(String msg) {
   SipClientConnection sc = null;
   try {
      sc = (SipClientConnection) Connector.open("sip:sippy.tester@host.com:5060");
      sc.initRequest("MESSAGE", null);
      sc.setHeader("Subject", "testing...");
      // write message body
      sc.setHeader("Content-Type", "text/plain");
      sc.setHeader("Content-Length", Integer.toString(msg.length()));
      OutputStream os = sc.openContentOutputStream();
      os.write(msg.getBytes());
      os.close(); // close stream and send the message
      // wait maximum 15 seconds for response
      boolean ok = sc.receive(15000);
      if(ok) {  // response received
          if(sc.getStatusCode() == 200) {
             // handle 200 OK response
          } else {
             // handle possible error responses
          }
      }
      sc.close();
    } catch(Exception ex) {
        // handle Exceptions
    }
}
```

Following class shows the same example using callback listener interface:

```
public class SipClient implements SipClientConnectionListener {
   public void sendTextMessage(String msg) {
      SipClientConnection sc = null;
      try {
         sc = (SipClientConnection) Connector.open("sip:sippy.tester@host.com:5060");
         sc.setListener(this);
         sc.initRequest("MESSAGE", null);
         sc.setHeader("Subject", "testing...");
         sc.setHeader("Content-Type", "text/plain");
         sc.setHeader("Content-Length", Integer.toString(msg.length()));
         OutputStream os = sc.openContentOutputStream();
         os.write(msg.getBytes());
         os.close(); // close stream and send message
      } catch(Exception ex) {
        // handle Exceptions
      }
     return;
   }

   public void notifyResponse(SipClientConnection scn) {
      try {
         // retrieve the response received
         sc.receive(0); // does not block response is there
         if(sc.getStatusCode() == 200) {
            // handle 200 OK response
         } else {
            // handle possible error responses
         }
         sc.close();
      } catch(Exception ex) {
          // handle Exceptions
      }
   }
}
```

See Also: `SipClientConnectionListener`[31], `SipConnectionNotifier`[28], `SipServerConnection`[24], `SipDialog.getNewClientConnection(String)`[35]

Member Summary

Methods

int	enableRefresh(SipRefreshListener srl)[23]
void	initAck()[20]
SipClientConnection	initCancel()[21]
void	initRequest(java.lang.String method, SipConnectionNotifier scn)[19]
boolean	receive(long timeout)[22]
void	setCredentials(java.lang.String username, java.lang.String password, java.lang.String realm)[23]
void	setListener(SipClientConnectionListener sccl)[22]
void	setRequestURI(java.lang.String URI)[20]

Inherited Member Summary

Methods inherited from interface `Connection`

`close()`

Methods inherited from interface `SipConnection`[9]

addHeader(String, String)[13], getDialog()[15], getHeader(String)[14], getHeaders(String)[13], getMethod()[14], getReasonPhrase()[14], getRequestURI()[14], getStatusCode()[14], openContentInputStream()[15], openContentOutputStream()[15], removeHeader(String)[13], send()[12], setHeader(String, String)[12]

Methods

initRequest(String, SipConnectionNotifier)

Declaration:

```
public void initRequest(java.lang.String method,
            javax.microedition.sip.SipConnectionNotifier[28] scn)
            throws IllegalArgumentException, SipException
```

Description:

Initializes `SipClientConnection` to a specific SIP request method (REGISTER, INVITE, MESSAGE, ...). The methods are defined in the RFC 3261 [1] and extensional specifications from SIP WG and SIMPLE WG [2][3]. The default RequestURI and headers will be initialized automatically. After this the `SipClientConnection` is in *Initialized* state.

The headers `From, Contact` are set according to the properties of `SipConnectionNotifier` given as the second parameter. If `SipConnectionNotifier` is `null` headers `From, Contact` are not set.

Headers that will be initialized are as follows:

```
To           // To address constructed from SIP URI given in Connector.open()
From         // Set by the system. If SipConnectionNotifier is given and the
                SIP identity is shared the value will be set according the
                terminal SIP settings (see SIP Identity
                for more details). If the SipConnectionNotifier is not given
                (= null) and/or the SIP identity is not shared the From
                header must be set to a default value e.g. anonymous URI
                (see RFC 3261 [1], chapter 8.1.1.3 From).
CSeq         // Set by the system
Call-ID      // Set by the system
Max-Forwards // Set by the system
Via          // Set by the system
Contact      // Set by the system (if SipConnectionNotifier is given) for
                REGISTER, INVITE and SUBSCRIBE. The value will be set
                according to the terminal IP settings and the
                SipConnectionNotifier properties. So the new request
                is associated with the SipConnectionNotifier. The
                other end can use the contact for subsequent requests.
```

Reference RFC 3261 [1] p.35 (8.1.1 Generating the Request) and p.159 (20 Header Fields)

Parameters:

`method` - Name of the method

`scn` - `SipConnectionNotifier` to which the request will be associated. If `SipConnectionNotifier` is `null` the request will not be associated to a user defined listening point.

Throws:

`java.lang.IllegalArgumentException` - if the method is invalid

`SipException`[53] - INVALID_STATE if the request can not be set, because of wrong state in `SipClientConnection`. Furthermore, ACK and CANCEL methods can not be initialized in *Created* state.

See Also: `SipConnectionNotifier`[28]

setRequestURI(String)

Declaration:

```
public void setRequestURI(java.lang.String URI)
            throws IllegalArgumentException, SipException
```

Description:

Sets Request-URI explicitly. Request-URI can be set only in *Initialized* state.

Parameters:

`URI` - Request-URI

Throws:

`java.lang.IllegalArgumentException` - MAY be thrown if the URI is invalid

`SipException`[53] - INVALID_STATE if the Request-URI can not be set, because of wrong state, INVALID_OPERATION if the Request-URI is not allowed to be set.

initAck()

Declaration:

```
public void initAck()
            throws SipException
```

Description:
Convenience method to initialize `SipClientConnection` with SIP request method ACK. ACK can be applied only to INVITE request. The method is available when final response (2xx) has been received. The header fields of the ACK are constructed in the same way as for any request sent within a dialog with the exception of the CSeq and the header fields related to authentication (RFC 3261 [1], p.82). The Request-URI and headers will be initialized automatically. After this the `SipClientConnection` is in *Initialized* state.

At least Request-URI and following headers will be set by the method (RFC 3261 [1] 12.2.1.1 Generating the Request p.73 and 8.1.1 Generating the Request p.35).
See also RFC 3261 [1] 13.2.2.4 2xx Responses (p.82)

```
Request-URI  // system uses the remote target and route set to build the Request-URI
To           // remote URI from the dialog state + remote tag of the dialog ID
From         // local URI from the dialog state + local tag of the dialog ID
CSeq         // the sequence number of the CSeq header field MUST be the same as
                the INVITE being acknowledged, but the CSeq method MUST be ACK.
Call-ID      // Call-ID of the dialog
Via          // Via header field indicates the transport used for the transaction
                and identifies the location where the response is to be sent
Route        // system uses the remote target and route set (if present)
                to build the Route header
Contact      // SHOULD include a Contact header field in any target refresh
                requests within a dialog, and unless there is a need to change
                it, the URI SHOULD be the same as used in previous requests within
                the dialog
Max-Forwards // header field serves to limit the number of hops a request can
                transit on the way to its destination.
```

The following rules also apply:
a) For error responses (3xx-6xx) the ACK is sent automatically by the system in transaction level. If user initializes an ACK which has already been sent an Exception will be thrown.
b) Using `initRequest("ACK", null)` builds the request from scratch and does not set the headers according to the current SIP dialog.

Throws:
`SipException`[53] - INVALID_STATE if the request can not be set, because of wrong state, INVALID_OPERATION if the ACK request can not be initialized for other reason (already sent or the original request is non-INVITE).

initCancel()

Declaration:
```
public javax.microedition.sip.SipClientConnection[16] initCancel()
            throws SipException
```

Description:
Convenience method to initialize `SipClientConnection` with SIP request method CANCEL. The method is available when a provisional response has been received. The CANCEL request starts a new transaction, that is why the method returns a new `SipClientConnection`. The CANCEL request will be built according to the original INVITE request within this connection. The RequestURI and headers will be initialized automatically. After this the `SipClientConnection` is in *Initialized* state. The message is ready to be sent.

The following information will be set by the method:

```
Request-URI  // copy from original request
To           // copy from original request
From         // copy from original request
CSeq         // same value for the sequence number as was present in the original
                request, but the method parameter MUST be equal to “CANCEL”
Call-ID      // copy from original request
Via          // single value equal to the top Via header field of the request
                being cancelled
Route        // If the request being cancelled contains a Route header field, the
                CANCEL request MUST include that Route header field's values
Max-Forwards // header field serves to limit the number of hops a request can
                transit on the way to its destination.
```

Reference RFC 3261 [1] p.53-54

Note: using `initRequest(“CANCEL”, null);` builds the request from scratch and does not set the headers according to the current SIP dialog.

A CANCEL request SHOULD NOT be sent to cancel a request other than INVITE.

Returns: A new `SipClientConnection` with preinitialized CANCEL request.

Throws:

`SipException`[53] - INVALID_STATE if the request can not be set, because of wrong state (in `SipClientConnection`) or the system has already got the 200 OK response (even if not read with `receive()` method). INVALID_OPERATION if CANCEL method can not be applied to the current request method.

receive(long)

Declaration:

```
public boolean receive(long timeout)
            throws SipException, IOException
```

Description:

Receives SIP response message. The receive method will update the `SipClientConnection` with the last new received response. If no message is received the method will block until something is received or specified amount of time has elapsed.

Parameters:

`timeout` - the maximum time to wait in milliseconds. 0 = do not wait, just poll

Returns: Returns `true` if response was received. Returns `false` if the given timeout elapsed and no response was received.

Throws:

`java.io.IOException` - if the message could not be received or because of network failure

`SipException`[53] - INVALID_STATE if the receive can not be called because of wrong state.

See Also: `SipConnection.send()`[12]

setListener(SipClientConnectionListener)

Declaration:

```
public void setListener(javax.microedition.sip.SipClientConnectionListener[31] sccl)
            throws IOException
```

Description:

Sets the listener for incoming responses. If a listener is already set it will be overwritten. Setting listener to `null` will remove the current listener.

Parameters:
`sccl` - reference to the listener object. Value `null` will remove the existing listener.

Throws:
`java.io.IOException` - if the connection is closed

enableRefresh(SipRefreshListener)

Declaration:
```
public int enableRefresh(javax.microedition.sip.SipRefreshListener[52] srl)
            throws SipException
```

Description:
Enables the refresh on for the request to be sent. The method return a refresh ID, which can be used to update or stop the refresh.

Parameters:
`srl` - callback interface for refresh events, if this is `null` the method returns 0 and refresh is not enabled.

Returns: refresh ID. If the request is not refreshable returns 0.

Throws:
`SipException`[53] - INVALID_STATE if the refresh can not be enabled in this state.

See Also: `SipRefreshHelper`[48]

setCredentials(String, String, String)

Declaration:
```
public void setCredentials(java.lang.String username, java.lang.String password,
            java.lang.String realm)
            throws SipException
```

Description:
Sets credentials for possible digest authentication. The application can set multiple credential triplets (username, password, realm) for one `SipClientConnection`. The username and password are specified for certain protection domain, which is defined by the realm parameter.

The credentials can be set:

- before sending the original request in *Initialized* state. The API implementation caches the credentials for later use.
- when 401 (Unauthorized) or 407 (Proxy Authentication Required) response is received in the *Unauthorized* state. The API implementation uses the given credentials to re-originate the request with proper authorization header. After that the `SipClientConnection` will be in *Proceeding* state.

See also `SipClientConnection` state chart.

Parameters:
`username` - username (for this protection domain)

`password` - user password (for this protection domain)

`realm` - defines the protection domain

Throws:
`java.lang.NullPointerException` - if the username, password or realm is null

java.lang.SipException - INVALID_STATE if the credentials can not be set in this state.

`SipException`[53]

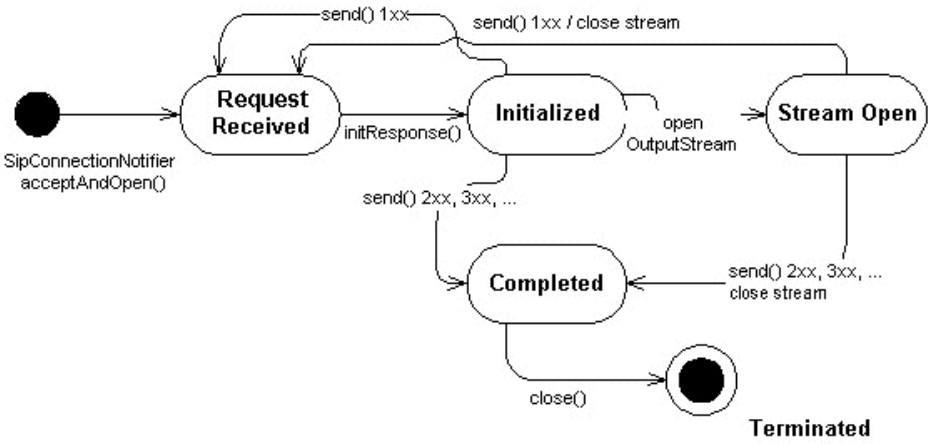
javax.microedition.sip
SipServerConnection

Declaration
```
public interface SipServerConnection extends SipConnection9
```

All Superinterfaces: `javax.microedition.io.Connection`, `SipConnection9`

Description
`SipServerConnection` represents SIP server transaction. `SipServerConnection` is created by the `SipConnectionNotifier` when a new request is received.

The `SipServerConnection` has following state chart:



- *Request Received*, `SipServerConnection` returned from `SipConnectionNotifier` or provisional response(s) (1xx) sent.
- *Initialized*, response initialized calling `initResponse()`
- *Stream Open*, `OutputStream` opened with `openContentOutputStream()`. Opening `InputStream` for received request does not trigger state transition.
- *Completed*, transaction completed with sending final response (2xx, 3xx, 4xx, 5xx, 6xx)
- *Terminated*, the final state, in which the SIP connection has been terminated by error or closed

Note: The state chart of SipServerConnection differs from the state chart of SIP server transaction, which can be found in RFC 3261 [1] p.136-140

Following methods are accessible in each state.

- *Request Received*
  `initResponse`
  `openContentInputStream`

- *Initialized*
  `addHeader`
  `setHeader`
  `removeHeader`
  `setReasonPhrase`
  `openContentOutputStream`
  `send`

- *Stream Open*
  Methods in `OutputStream` and `SipConnection.send`
- *Completed*
  All get-methods, see below
- *Terminated*
  The transaction and this connection is closed. The I/O methods above will throw IOException

Following methods can be called in every state. The functionality is defined by the method depending of the information availability.

- *Can be called in every state*
  `getHeader`
  `getHeaders`
  `getRequestURI`
  `getMethod`
  `getStatusCode`
  `getReasonPhrase`
  `getDialog`
  `close` // causes state transition to *Terminated* state

Code Examples

Following code example illustrates the usage of SIP server connection: opening, receiving one request (MESSAGE) and sending response:

```
public receiveMessage() {
   SipConnectionNotifier scn = null;
   SipServerConnection ssc = null;
   String method = null;

   try {
      // Open SIP server connection and listen to port 5060
      scn = (SipConnectionNotifier) Connector.open("sip:5060");

      // block and wait for incoming request.
      // SipServerConnection is establised and returned
      // when new request is received.
      ssc = scn.acceptAndOpen();
     // what was the SIP method
     method = ssc.getMethod();
     if(method.equals("MESSAGE")) {
        // read the content of the MESSAGE
        String contentType = ssc.getHeader("Content-Type");
        if((contentType != null) && contentType.equals("text/plain")) {
           InputStream is = ssc.openContentInputStream();
           int ch;
           // read content
           while ((ch = is.read()) != -1) {
            ...
           }
        }
        // initialize SIP 200 OK and send it back
        ssc.initResponse(200);
        ssc.send();
        ssc.close();
     }
   } catch(Exception ex) {
    // handle IOException, InterruptedIOException, SecurityException
    // or SipException
   }
}
```

See Also: SipConnectionNotifier[28], SipServerConnectionListener[32], SipClientConnection[16]

Member Summary

Methods

void	initResponse(int code)[27]
void	setReasonPhrase(java.lang.String phrase)[27]

Inherited Member Summary

Methods inherited from interface `Connection`

`close()`

Methods inherited from interface SipConnection[9]

Inherited Member Summary
addHeader(String, String)[13], getDialog()[15], getHeader(String)[14], getHeaders(String)[13], getMethod()[14], getReasonPhrase()[14], getRequestURI()[14], getStatusCode()[14], openContentInputStream()[15], openContentOutputStream()[15], removeHeader(String)[13], send()[12], setHeader(String, String)[12]

Methods

initResponse(int)

Declaration:

```
public void initResponse(int code)
            throws IllegalArgumentException, SipException
```

Description:

Initializes `SipServerConnection` with a specific SIP response to the received request. The default headers and reason phrase will be initialized automatically. After this the `SipServerConnection` is in *Initialized* state. The response can be sent.

The procedure of generating the response and header fields is defined in RFC 3261 [1] p. 49-50. At least following information is set by the method:

```
From     // MUST equal the From header field of the request
Call-ID  // MUST equal the Call-ID header field of the request
CSeq     // MUST equal the CSeq field of the request
Via      // MUST equal the Via header field values in the request
            and MUST maintain the same ordering
To       // MUST Copy if exists in the original request,
            'tag' MUST be added if not present
```

Furthermore, if the system has automatically sent the “100 Trying” response, the 100 response initialized and sent by the user is just ignored.

Parameters:

`code` - Response status code 1xx - 6xx

Throws:

`java.lang.IllegalArgumentException` - if the status code is out of range 100-699 (RFC 3261 p.28-29)

`SipException`[53] - INVALID_STATE if the response can not be initialized, because of wrong state.

setReasonPhrase(String)

Declaration:

```
public void setReasonPhrase(java.lang.String phrase)
            throws SipException, IllegalArgumentException
```

Description:

Changes the default reason phrase.

Throws:

`java.lang.IllegalArgumentException` - if the reason phrase is illegal.

`SipException`[53] - INVALID_STATE if the response can not be initialized, because of wrong state. INVALID_OPERATION if the reason phrase can not be set.

javax.microedition.sip
SipConnectionNotifier

Declaration
`public interface` **`SipConnectionNotifier extends javax.microedition.io.Connection`**

All Superinterfaces: `javax.microedition.io.Connection`

Description
This interface defines a SIP server connection notifier. The SIP server connection is opened with `Connector.open()`using a generic SIP URI string with the host and user omitted. For example, URI `sip:5060` defines an inbound SIP server connection on port 5060. The local address can be discovered using the `getLocalAddress()` method. If the port number is already reserved the `Connector.open()` MUST throw `IOException`.

`SipConnectionNotifier` can be also opened with `sips:` protocol scheme, which indicates that this server connection accepts only requests over secure transport (as defined in RFC 3261 [1] for SIPS URIs).

`SipConnectionNotifier` is queueing received messages. In order to receive incoming requests application calls the `acceptAndOpen()` method, which returns a `SipServerConnection` instance. If there are no messages in the queue `acceptAndOpen()` will block until a new request is received. `SipServerConnection` holds the incoming SIP request message. `SipServerConnection` is used to initialize and send responses.

Access to SIP server connections may be restricted by the security policy of the device. `Connector.open` `MUST` check access for the initial SIP server connection and `acceptAndOpen()` MUST check before returning each new `SipServerConnection`.

A SIP server connection can be used to dynamically select an available port by omitting both the host and the port parameters in the connection SIP URI string. The string `sip:` defines an inbound SIP server connection on a port which is allocated by the system. To discover the assigned port number use the `getLocalPort()` method.

The `SipConnectionNotifier` offers also an asynchronous callback interface to wait for incoming requests. The interface is defined in `SipServerConnectionListener` , which the user has to implement in order to receive notifications about incoming requests.

Here is an example method, which opens an inbound SIP server connection:

```
public SipServerConnection openSipServerConnection() {
   SipConnectionNotifier scn = null;
   SipServerConnection ssc = null;

   try {
      scn = (SipConnectionNotifier) Connector.open(“sip:”); //
let the system select the port
      ssc = scn.acceptAndOpen();
      return(ssc);
   } catch(Exception ex) {
     // handle IOException, InterruptedIOException, SecurityException
     // or SipException
   }
}
```

Here is another example class, which uses the callback listener interface:

```
public class SipServer implements SipServerConnectionListener {
   SipServer(String uri) {
      try {
         scn = (SipConnectionNotifier) Connector.open("sip:5080"); //
user selects the port
         scn.setListener(this);
      } catch(Exception ex) {
         // handle Exceptions
      }
   }

   public void notifyRequest(SipConnectionNotifier scn) {
      SipServerConnection ssc = scn.acceptAndOpen();
      String method = ssc.getMethod();
      // continue with the incoming request etc...
   }
}
```

See Also: SipConnection[9], SipServerConnection[24], SipClientConnection[16], SipServerConnectionListener[32]

Member Summary	
Methods	
SipServerConnection	acceptAndOpen()[29]
java.lang.String	getLocalAddress()[30]
int	getLocalPort()[30]
void	setListener(SipServerConnectionListener sscl)[30]

Inherited Member Summary
Methods inherited from interface Connection
close()

Methods

acceptAndOpen()

Declaration:

```
public javax.microedition.sip.SipServerConnection[24] acceptAndOpen()
            throws IOException, InterruptedIOException, SipException
```

Description:

Accepts and opens a new SipServerConnection in this listening point. If there are no messages in the queue method will block until a new request is received.

Returns: SipServerConnection which carries the received request

Throws:

java.io.IOException - if the connection can not be established

`java.io.InterruptedIOException` - if the connection is closed

`SipException`[53] - TRANSACTION_UNAVAILABLE if the system can not open new SIP transactions.

setListener(SipServerConnectionListener)

Declaration:

```
public void setListener(javax.microedition.sip.SipServerConnectionListener[32] sscl)
            throws IOException
```

Description:

Sets a listener for incoming SIP requests. If a listener is already set it will be overwritten. Setting listener to `null` will remove the current listener.

Throws:

`java.io.IOException` - if the connection was closed

See Also: `SipServerConnectionListener`[32], `SipServerConnection`[24]

getLocalAddress()

Declaration:

```
public java.lang.String getLocalAddress()
            throws IOException
```

Description:

Gets the local IP address for this SIP connection.

Returns: local IP address. Returns `null` if the address is not available.

Throws:

`java.io.IOException` - if the connection was closed

getLocalPort()

Declaration:

```
public int getLocalPort()
            throws IOException
```

Description:

Gets the local port for this SIP connection.

Returns: local port number, that the notifier is listening to. Returns 0 if the port is not available.

Throws:

`java.io.IOException` - if the connection was closed

javax.microedition.sip
SipClientConnectionListener

Declaration
`public interface` **`SipClientConnectionListener`**

Description
Listener interface for incoming SIP responses.

See Also: `SipClientConnection`[16]

Member Summary	
Methods	
`void`	`notifyResponse(SipClientConnection scc)`[31]

Methods

notifyResponse(SipClientConnection)

Declaration:
`public void` **`notifyResponse`**`(javax.microedition.sip.SipClientConnection`[16] `scc)`

Description:
This method gives the `SipClientConnection` instance, which has received a new SIP response. The application implementing this listener interface has to call `SipClientConnection.receive()` to initialize the `SipClientConnection` object with the new response.

Parameters:
`scc` - `SipClientConnection` carrying the response

javax.microedition.sip
SipServerConnectionListener

Declaration
```
public interface SipServerConnectionListener
```

Description
Listener interface for incoming SIP requests.

See Also: `SipConnectionNotifier`[28]

Member Summary	
Methods	
`void`	`notifyRequest(SipConnectionNotifier ssc)`[32]

Methods

notifyRequest(SipConnectionNotifier)

Declaration:
```
public void notifyRequest(javax.microedition.sip.SipConnectionNotifier[28] ssc)
```

Description:
This method will notify the listener that a new request is received. This method gives the `SipConnectionNotifier` instance. The user has to call the `SipConnectionNotifier.acceptAndOpen()` to get the `SipServerConnection` object that holds the server transaction and the request received.

Parameters:
`scn` - `SipConnectionNotifier` carrying `SipServerConnection`

javax.microedition.sip
SipDialog

Declaration
`public interface SipDialog`

Description
`SipDialog` represents one SIP Dialog. The `SipDialog` can be retrieved from a `SipConnection` object, when it is available (at earliest after provisional 101-199 response).

Basically, two SIP requests can open a dialog:

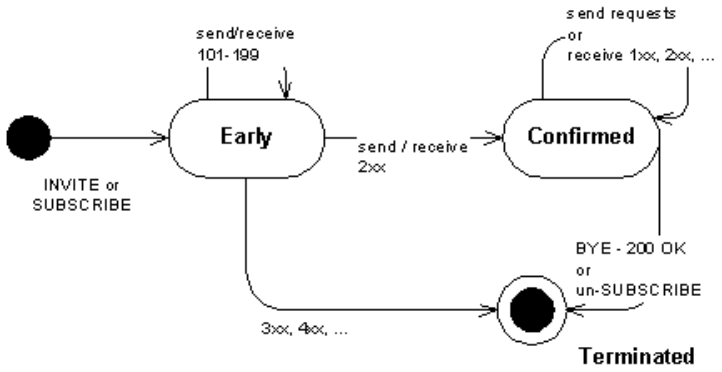
- INVITE-1xx-2xx-ACK will open a dialog. Subsequent `SipClientConnection` in the same dialog can be obtained by calling `getNewClientConnection(String method)` method. The dialog is terminated when the transaction BYE-200 OK is completed. For more information please refer to RFC 3261 [1], Chapter 12.

- SUBSCRIBE-200 OK(or matching NOTIFY) will open a dialog. Subsequent `SipClientConnection` in the same dialog can be obtained by calling `getNewClientConnection(String method)` method. The dialog is terminated when a notifier sends a NOTIFY request with a “Subscription-State” of “terminated” and there is no other subscriptions alive with this dialog. For more information please refer to RFC 3265 [2], Chapter 3.3.4.

`SipDialog` has following states (for both client and server side):

- *early (and created)*, provisional 101-199 response received (or sent)
  Method `getNewClientConnection()` can not be called in this state.
- *confirmed*, final 2xx response received (or sent)
  All methods available.
- *terminated*, no response or error response (3xx-6xx) received (or sent). Also if the dialog is terminated with BYE or un-SUBSCRIBE.
  Method `getNewClientConnection()` can not be called in this state.

The `SipDialog` has following state chart:



Following code example shows a simple example of using `SipDialog` in conjunction with `SipClientConnection` and `SipServerConnection`. `SipDialog` is used to send subsequent SUBSCRIBE request as well as detecting if received NOTIFY belongs to the same dialog (i.e. subscription). Further dialog information like "Call-ID" or "remote tag" can be read from the subsequent `SipClientConnection` as demonstrated.

```
class SipDialogExample implements SipServerConnectionListener {
   SipDialog dialog;
   SipClientConnection scc;
   SipConnectionNotifier scn;
   String callID;
   String remoteTag;
   public void sendSubscribe() {
       try {
          scn = (SipConnectionNotifier) Connector.open("sip:");
          scn.setListener(this);
          scc = (SipClientConnection)
        Connector.open("sip:sippy.user@host.com");
          scc.initRequest("SUBSCRIBE", scn);
          scc.setHeader("Accept", "application/xpidf+xml");
          scc.setHeader("Event", "presence");
          scc.setHeader("Expires", "950");
          scc.send();
          boolean resp = scc.receive(10000); // wait 10 secs for response
          if(resp) {
              if(scc.getStatusCode() == 200) {
                  dialog = scc.getDialog();
                  // initialize new SipClientConnection
                  scc = dialog.getNewClientConnection("SUBSCRIBE");
                  // read dialog Call-ID
                  callID = scc.getHeader("Call-ID");
                  // read remote tag
                  SipHeader sh = new SipHeader("To", scc.getHeader("To"));
                  remoteTag = sh.getParameter("tag");
                  // unSUBSCRIBE
                  scc.setHeader("Expires", "0");
                  scc.send();
              }
          } else {
              // didn't receive any response in given time
          }
       } catch(Exception ex) {
           // handle Exceptions
       }
   }
   public void notifyRequest(SipConnectionNotifier scn) {
       try {
          SipServerConnection ssc;
          // retrieve the request received
          ssc = scn.acceptAndOpen();
          // check if the received request is NOTIFY and it belongs to our dialog
          if(ssc.getMethod().equals("NOTIFY") && dialog.isSameDialog(ssc)) {
              ssc.initResponse(200);
              ssc.send();
          }else {
              // send 481 "Subscription does not exist"
              ssc.initResponse(481);
              ssc.send();
          }
       } catch(Exception ex) {
           // handle Exceptions
       }
   }
}
```

See Also: `SipConnection.getDialog()`[15]

Member Summary

Fields

static byte	`CONFIRMED`[35]
static byte	`EARLY`[35]
static byte	`TERMINATED`[35]

Member Summary	
Methods	
java.lang.String	getDialogID()[36]
SipClientConnection	getNewClientConnection(java.lang.String method)[35]
byte	getState()[36]
boolean	isSameDialog(SipConnection sc)[36]

Fields

EARLY

Declaration:
```
public static final byte EARLY
```

CONFIRMED

Declaration:
```
public static final byte CONFIRMED
```

TERMINATED

Declaration:
```
public static final byte TERMINATED
```

Methods

getNewClientConnection(String)

Declaration:
```
public javax.microedition.sip.SipClientConnection[16]
            getNewClientConnection(java.lang.String method)
            throws IllegalArgumentException, SipException
```

Description:
Returns new `SipClientConnection` in this dialog. The `SipClientConnection` will be pre-initialized with the given method and following headers will be set at least (for details see RFC 3261 [1] 12.2.1.1 Generating the Request, p.73):

```
To
From
CSeq
Call-ID
Max-Forwards
Via
Contact
Route        // if the dialog route is not empty
```

Returns: `SipClientConnection` with preset headers.

Throws:
`java.lang.IllegalArgumentException` - if the method is invalid

`SipException`[53] - INVALID_STATE if the new connection can not be established in the current state of dialog.

isSameDialog(SipConnection)

Declaration:
```
public boolean isSameDialog(javax.microedition.sip.SipConnection[9] sc)
```

Description:
Does the given `SipConnection` belong to this dialog.

Parameters:
`sc` - `SipConnection` to be checked, can be either `SipClientConnection` or `SipServerConnection`

Returns: `true` if the `SipConnection` belongs to the this dialog. Returns `false` if the connection is not part of this dialog or the dialog is terminated.

getState()

Declaration:
```
public byte getState()
```

Description:
Returns the state of the SIP Dialog.

Returns: dialog state byte number.

getDialogID()

Declaration:
```
public java.lang.String getDialogID()
```

Description:
Returns the ID of the SIP Dialog.

Returns: Dialog ID (Call-ID + remote tag + local tag). Returns `null` if the dialog is terminated.

javax.microedition.sip
SipHeader

Declaration

```
public class SipHeader

java.lang.Object
  |
  +--javax.microedition.sip.SipHeader
```

Description

`SipHeader` provides generic SIP header parser helper. This class can be used to parse bare String header values that are read from SIP message using e.g. `SipConnection.getHeader()` method. It should be noticed that `SipHeader` is separate helper class and not mandatory to use for creating SIP connections. Correspondingly, SIP headers can be constructed with this class.
`SipHeader` uses generic format to parse the header value and parameters following the syntax given in RFC 3261[1] p.31

- `field-name: field-value *(;parameter-name=parameter-value)`

`SipHeader` also supports parsing for the following authorization and authentication headers: `WWW-Authenticate, Proxy-Authenticate, Proxy-Authorization, Authorization`. For those a slightly different syntax is applied, where comma is used as the parameter separator instead of semicolon. Authentication parameters are accessible through `get/set/removeParameter()` methods in the same way as generic header parameters.

- `auth-header-name: auth-scheme LWS auth-param *(COMMA auth-param)`

The ABNF for the `SipHeader` parser is derived from SIP ABNF as follows:

```
header          =  header-name ":" header-value *(";" generic-param) /
                   WWW-Authenticate / Proxy-Authenticate /
                   Proxy-Authorization / Authorization
header-name     =  token
generic-param   =  token [ EQUAL gen-value ]
gen-value       =  token / host / quoted-string
header-value    =  1*(chars) / name-addr
chars           =  %x20-3A / "=" / %x3F-7E
                   ; any visible character except ";" "<" ">"
```

Reference, SIP 3261 [1] p.159 Header Fields and p.219 SIP BNF for terminals not defined in this BNF.

Example headers:

`Call-ID: a84b4c76e66710`

Call-ID header with no parameters

`From: Bob <sip:bob@biloxi.com>;tag=a6c85cf`

From header with parameter 'tag'

`Contact: <sip:alice@pc33.atlanta.com>`

Contact header with no parameters

```
Via: SIP/2.0/UDP pc33.atlanta.com;branch=z9hG4bKhjhs8ass877
```

Via header with parameter 'branch'

```
Contact: "Mr. Watson" <sip:watson@worcester.bell-
telephone.com>;q=0.7;expires=3600
```

Contact header with parameters 'q' and 'expires'

```
WWW-Authenticate: Digest realm="atlanta.com", domain="sip:boxesbybob.com",
qop="auth", nonce="f84f1cec41e6cbe5aea9c8e88d359", opaque="", stale=FALSE,
algorithm=MD5
```

WWW-Authenticate header with digest authentication scheme.

See Also: SipAddress[42]

Member Summary

Constructors

	SipHeader(java.lang.String name, java.lang.String headerValue)[38]

Methods

java.lang.String	getHeaderValue()[39]
java.lang.String	getName()[39]
java.lang.String	getParameter(java.lang.String name)[40]
java.lang.String[]	getParameterNames()[40]
java.lang.String	getValue()[39]
void	removeParameter(java.lang.String name)[41]
void	setName(java.lang.String name)[39]
void	setParameter(java.lang.String name, java.lang.String value)[40]
void	setValue(java.lang.String value)[40]
java.lang.String	toString()[41]

Inherited Member Summary

Methods inherited from class `Object`

`equals(Object), getClass(), hashCode(), notify(), notifyAll(), wait(), wait(), wait()`

Constructors

SipHeader(String, String)

Declaration:

```
public SipHeader(java.lang.String name, java.lang.String headerValue)
            throws IllegalArgumentException
```

Description:
Constructs a `SipHeader` from name value pair. For example:

name = `Contact`

value = `<sip:UserB@192.168.200.201>;expires=3600`

Parameters:
`name` - name of the header (Contact, Call-ID, ...)

`headerValue` - full header value as String

Throws:
`java.lang.IllegalArgumentException` - if the header value or name are invalid

Methods

setName(String)

Declaration:
```
public void setName(java.lang.String name)
            throws IllegalArgumentException
```

Description:
Sets the header name, for example `Contact`

Throws:
`java.lang.IllegalArgumentException` - if the name is invalid

getName()

Declaration:
```
public java.lang.String getName()
```

Description:
Returns the name of this header

Returns: the name of this header as String

getValue()

Declaration:
```
public java.lang.String getValue()
```

Description:
Returns the header value without header parameters. For example for header `<sip:UserB@192.168.200.201>;expires=3600` method returns `<sip:UserB@192.168.200.201>`
In the case of an authorization or authentication header `getValue()` returns only the authentication scheme e.g. “Digest”.

Returns: header value without header parameters

getHeaderValue()

Declaration:
```
public java.lang.String getHeaderValue()
```

Description:
Returns the full header value including parameters. For example "Alice <sip:alice@atlanta.com>;tag=1928301774"

Returns: full header value including parameters

setValue(String)

Declaration:
```
public void setValue(java.lang.String value)
            throws IllegalArgumentException
```

Description:
Sets the header value as String without parameters. For example "<sip:UserB@192.168.200.201>". The existing (if any) header parameter values are not modified. For the authorization and authentication header this method sets the authentication scheme e.g. "Digest".

Throws:
`java.lang.IllegalArgumentException` - if the value is invalid or there is parameters included.

getParameter(String)

Declaration:
```
public java.lang.String getParameter(java.lang.String name)
```

Description:
Returns the value of one header parameter. For example, from value "<sip:UserB@192.168.200.201>;expires=3600" the method call `getParameter("expires")` will return "3600".

Parameters:
`name` - name of the header parameter

Returns: value of header parameter. returns empty string for a parameter without value and null if the parameter does not exist.

getParameterNames()

Declaration:
```
public java.lang.String[] getParameterNames()
```

Description:
Returns the names of header parameters. Returns `null` if there are no header parameters.

Returns: names of the header parameters. Returns `null` if there are no parameters.

setParameter(String, String)

Declaration:
```
public void setParameter(java.lang.String name, java.lang.String value)
            throws IllegalArgumentException
```

Description:
Sets value of header parameter. If parameter does not exist it will be added. For example, for header value "<sip:UserB@192.168.200.201>" calling `setParameter("expires", "3600")` will

construct header value "<sip:UserB@192.168.200.201>;expires=3600".
If the value is null, the parameter is interpreted as a parameter without value.

Parameters:

`name` - name of the header parameter

`value` - value of the parameter

Throws:

`java.lang.IllegalArgumentException` - if the parameter name or value are invalid

removeParameter(String)

Declaration:

```
public void removeParameter(java.lang.String name)
```

Description:

Removes the header parameter, if it is found in this header.

Parameters:

`name` - name of the header parameter

toString()

Declaration:

```
public java.lang.String toString()
```

Description:

Returns the String representation of the header according to header type. For example:

- `From: Alice <sip:alice@atlanta.com>;tag=1928301774`
- `WWW-Authenticate: Digest realm="atlanta.com", domain="sip:boxesbybob.com", qop="auth", nonce="f84f1cec41e6cbe5aea9c8e88d359", opaque="", stale=FALSE, algorithm=MD5`

Overrides: `toString` in class `Object`

javax.microedition.sip
SipAddress

Declaration

```
public class SipAddress

java.lang.Object
  |
  +--javax.microedition.sip.SipAddress
```

Description

`SipAddress` provides a generic SIP address parser. This class can be used to parse either full SIP name addresses like: `BigGuy <sip:UserA@atlanta.com>` or SIP/SIPS URIs like: `sip:+13145551111@ss1.atlanta.com;user=phone` or `sips:alice@atlanta.com;transport=tcp`.

Correspondingly, valid SIP addresses can be constructed with this class.

`SipAddress` has following functional requirements:
- `SipAddress` does not escape address strings.
- `SipAddress` ignores headers part of SIP URI.
- `SipAddress` valid scheme format is the same as defined in SIP BNF for absolute URI.
- The valid Contact address `"*"` is accepted in `SipAddress`. In this case all properties will be null and port number is 0. Yet `toString()` method will return the value `"*"`.

Reference, SIP 3261 [1] p.153 Example SIP and SIPS URIs and p.228 `name-addr` in SIP BNF specification.

Member Summary

Constructors

	`SipAddress(java.lang.String address)`[43]
	`SipAddress(java.lang.String displayName, java.lang.String URI)`[43]

Methods

`java.lang.String`	`getDisplayName()`[44]
`java.lang.String`	`getHost()`[45]
`java.lang.String`	`getParameter(java.lang.String name)`[46]
`java.lang.String[]`	`getParameterNames()`[47]
`int`	`getPort()`[46]
`java.lang.String`	`getScheme()`[44]
`java.lang.String`	`getURI()`[45]
`java.lang.String`	`getUser()`[44]
`void`	`removeParameter(java.lang.String name)`[46]
`void`	`setDisplayName(java.lang.String name)`[44]
`void`	`setHost(java.lang.String host)`[45]
`void`	`setParameter(java.lang.String name, java.lang.String value)`[46]
`void`	`setPort(int port)`[46]
`void`	`setScheme(java.lang.String scheme)`[44]
`void`	`setURI(java.lang.String URI)`[45]
`void`	`setUser(java.lang.String user)`[45]

Member Summary
java.lang.String toString()47

Inherited Member Summary
Methods inherited from class `Object` `equals(Object), getClass(), hashCode(), notify(), notifyAll(), wait(), wait(), wait()`

Constructors

SipAddress(String)

Declaration:

```
public SipAddress(java.lang.String address)
            throws IllegalArgumentException
```

Description:

Construct a new `SipAddress` from string. The string can be either name address:

```
Display Name <sip:user:password@host:port;uri-parameters>
```

or plain SIP URI:

```
sip:user:password@host:port;uri-parameters
```

Parameters:

`address` - SIP address as String

Throws:

`java.lang.IllegalArgumentException` - if there was an error in parsing the address.

SipAddress(String, String)

Declaration:

```
public SipAddress(java.lang.String displayName, java.lang.String URI)
            throws IllegalArgumentException
```

Description:

Construct a new `SipAddress` from display name and URI.

Parameters:

`displayName` - user display name

`URI` - SIP URI

Throws:

`java.lang.IllegalArgumentException` - if there was an error in parsing the arguments.

Methods

getDisplayName()

Declaration:
```
public java.lang.String getDisplayName()
```

Description:
Returns the display name of SIP address.

Returns: display name or null if not available

setDisplayName(String)

Declaration:
```
public void setDisplayName(java.lang.String name)
            throws IllegalArgumentException
```

Description:
Sets the display name. Empty string “” removes the display name.

Throws:
`java.lang.IllegalArgumentException` - if the display name is invalid

getScheme()

Declaration:
```
public java.lang.String getScheme()
```

Description:
Returns the scheme of SIP address.

Returns: the scheme of this SIP address e.g. `sip` or `sips`

setScheme(String)

Declaration:
```
public void setScheme(java.lang.String scheme)
            throws IllegalArgumentException
```

Description:
Sets the scheme of SIP address. Valid scheme format is defined in RFC 3261 [1] p.224

Throws:
`java.lang.IllegalArgumentException` - if the scheme is invalid

getUser()

Declaration:
```
public java.lang.String getUser()
```

Description:
Returns the user part of SIP address.

Returns: user part of SIP address. Returns `null` if the user part is missing.

setUser(String)

Declaration:

```
public void setUser(java.lang.String user)
            throws IllegalArgumentException
```

Description:
Sets the user part of SIP address.

Throws:
`java.lang.IllegalArgumentException` - if the user part is invalid

getURI()

Declaration:

```
public java.lang.String getURI()
```

Description:
Returns the URI part of the address (without parameters) i.e. `scheme:user@host:port`.

Returns: the URI part of the address

setURI(String)

Declaration:

```
public void setURI(java.lang.String URI)
            throws IllegalArgumentException
```

Description:
Sets the URI part of the SIP address (without parameters) i.e. `scheme:user@host:port`. Possible URI parameters are ignored.

Throws:
`java.lang.IllegalArgumentException` - if the URI is invalid

getHost()

Declaration:

```
public java.lang.String getHost()
```

Description:
Returns the host part of the SIP address.

Returns: host part of this address.

setHost(String)

Declaration:

```
public void setHost(java.lang.String host)
            throws IllegalArgumentException
```

Description:
Sets the host part of the SIP address.

Throws:
`java.lang.IllegalArgumentException` - if the host is invalid

getPort()

Declaration:
```
public int getPort()
```

Description:
Returns the port number of the SIP address. If port number is not set, return 5060. If the address is wildcard "*" return 0.

Returns: the port number

setPort(int)

Declaration:
```
public void setPort(int port)
            throws IllegalArgumentException
```

Description:
Sets the port number of the SIP address. Valid range is 0-65535, where 0 means that the port number is removed from the address URI.

Parameters:
`port` - port number, valid range 0-65535, 0 means that port number is removed from the address URI

Throws:
`java.lang.IllegalArgumentException` - if the port number is invalid

getParameter(String)

Declaration:
```
public java.lang.String getParameter(java.lang.String name)
```

Description:
Returns the value associated with the named URI parameter.

Parameters:
`name` - the name of the parameter

Returns: the value of the named parameter, or empty string for parameters without value and null if the parameter is not defined

setParameter(String, String)

Declaration:
```
public void setParameter(java.lang.String name, java.lang.String value)
            throws IllegalArgumentException
```

Description:
Sets the named URI parameter to the specified value. If the value is null the parameter is interpreted as a parameter without value. Existing parameter will be overwritten, otherwise the parameter is added.

Throws:
`java.lang.IllegalArgumentException` - if the parameter is invalid RFC 3261, chapter 19.1.1 SIP and SIPS URI Components "URI parameters" p.149

removeParameter(String)

Declaration:
```
public void removeParameter(java.lang.String name)
```

Description:
Removes the named URI parameter.

getParameterNames()

Declaration:
```
public java.lang.String[] getParameterNames()
```

Description:
Returns a String array of all parameter names.

Returns: String array of parameter names. Returns `null` if the address does not have any parameters.

toString()

Declaration:
```
public java.lang.String toString()
```

Description:
Returns a fully qualified SIP address, with display name, URI and URI parameters. If display name is not specified only a SIP URI is returned. If the port is not explicitly set (to 5060 or other value) it will be omitted from the address URI in returned String.

Overrides: `toString` in class `Object`

Returns: a fully qualified SIP name address, SIP or SIPS URI

javax.microedition.sip
SipRefreshHelper

Declaration

```
public class SipRefreshHelper

java.lang.Object
  |
  +--javax.microedition.sip.SipRefreshHelper
```

Description

This class implements the functionality that facilitates the handling of refreshing requests on behalf of the application. Some SIP requests (REGISTER, SUBSCRIBE, ...) need to be timely refreshed (binding between end point and server, see RFC 3261 - chapter 10.2.1 page 58). For example the REGISTER request (RFC 3261, chapter 10 page 56) needs to be re-sent to ensure that the originating end point is still well and alive. The request's validity is proposed by the end point in the request and confirmed in the response by the registrar/ notifier for example in expires header (RFC 3261, chapter 2 page 5). The handling of such binding would significantly increase application complexity and size. As a consequence the `SipRefreshHelper` can be used to facilitate such operations. When the application wants to send a refreshable request it:

- implements `SipRefreshListener` callback interface.
- creates a new `SipClientConnection` and sets it up.
- calls the method `enableRefresh(SipRefreshListener)`. A refresh ID is returned. If the request is not refreshable the method returns 0.
- if the refresh task fails a failure event is sent to the `SipRefreshListener`

A reference to the `SipRefreshHelper` object is obtained by calling the static method `SipRefreshHelper.getInstance()` (singleton pattern).

Finally, using the refresh ID returned from `enableRefresh(SipRefreshListener)` the application can:

- `stop()` a refresh. The possible binding between end point and server is cancelled (RFC3261, chapter 10.2.2 page 61).
- `update(...)` the refreshed request with new parameters (Contact info and new payload). Note that this functionality is limited to the most typical case. A more complex case would require to stop the refresh and to create a new request with the needed updates.

When all refresh tasks belonging to one refresh listener are stopped, the listener reference will be removed from the `SipRefreshHelper`.

Code example where REGISTER is sent, updated and finally stopped:

```
class SipRefreshExample implements SipClientConnectionListener,
 SipRefreshListener {
   int refreshID = 0;
   int refreshStatus = 0;
   SipRefreshHelper refHelper = null;
   public void sendRegister() {
       SipClientConnection sc = null;
       try {
          sc = (SipClientConnection)
        Connector.open("sip:sippy.user@host.com:5060");
          sc.setListener(this);
          sc.initRequest("REGISTER", null);
          sc.setHeader("Contact", "<sip:UserB@192.168.200.201>;expires=3600");
          sc.setHeader("Contact", "<mailto:UserB@biloxi.com>;expires=4294967295");
          refreshID = sc.enableRefresh(this);
          sc.send();
          refHelper = SipRefreshHelper.getInstance();
          //-----------------------------
          // do something else for a while
          //-----------------------------
          // update REGISTER, with new "mailto:" Contact and no content
          if(refreshStatus == 200) { // check that refresh was successful
              String c[] = { "<mailto:UserB@company.com>" };
              refHelper.update(refreshID, c, null, 0, 6000);
          }
          //-----------------------------
          // do something else for a while
          //-----------------------------
          // stop REGISTER refresh altogether
          if(refreshStatus == 200) { // check that refresh is still ok
              refHelper.stop(refreshID);
          }
       } catch(Exception ex) { // handle Exceptions
       }
   }
   public void notifyResponse(SipClientConnection scc) {
       try {
          // retrieve the response received
          scc.receive(0);
          if(scc.getStatusCode() == 200) {
             // handle 200 OK response
          }else {
             // handle possible error responses
          }
       } catch(Exception ex) {
           // handle Exceptions
       }
   }
   public void refreshEvent(int ID, int statusCode, String reasonPhrase) {
      refreshStatus = statusCode;
      if (statusCode == 0){
               // stopped refresh
      }else if (statusCode == 200){
               // successful refresh
      }else {
               // failed request
      }
   }
}
```

See Also: SipClientConnection.enableRefresh(SipRefreshListener)[23]

Member Summary

Methods

static SipRefreshHelper	getInstance()[50]
void	stop(int refreshID)[50]
java.io.OutputStream	update(int refreshID, java.lang.String contact, java.lang.String type, int length, int expires)[50]

Inherited Member Summary

Methods inherited from class `Object`

`equals(Object), getClass(), hashCode(), notify(), notifyAll(), toString(), wait(), wait(), wait()`

Methods

getInstance()

Declaration:
```
public static javax.microedition.sip.SipRefreshHelper[48] getInstance()
```

Description:
Returns the instance of `SipRefreshHelper`

Returns: the instance of `SipRefreshHelper` singleton

stop(int)

Declaration:
```
public void stop(int refreshID)
```

Description:
Stop refreshing a specific request related to refeshID. The possible binding between end point and registrar/ notifier is cancelled (RFC3261, chapter 10.2.2 page 61). An event will be sent to the listeners with refreshID and statusCode = 0 and reasonPhrase = “refresh stopped”.

Parameters:
`refreshID` - the ID of the refresh to be stopped. If the ID does not match any refresh task the method does nothing.

update(int, String[], String, int, int)

Declaration:
```
public java.io.OutputStream update(int refreshID, java.lang.String[] contact,
            java.lang.String type, int length, int expires)
```

Description:
Updates one refreshed request with new values.

- new `Contact` header values. existing values are kept if this is null.

- `Expires` header value. expires = 0 has the same effect as calling `stop(refreshID)`. expires = -1 leaves the `Expires` header out.

- new payload: `Content-Type` and `Content-Length`. The message is sent when the returned `OutputStream` is closed. If no content is set the message will be sent automatically and the method returns `null`.

Parameters:

`refreshID` - ID returned from `enableRefresh(...)`. If the ID does not match any refresh task the method just returns without doing anything.

`contact` - new Contact headers as String array. Replaces all old values. Multiple Contact header values are applicable only for REGISTER method. If contact param is null or empty the system will set the Contact header.

`type` - value of Content-Type (null or empty, no content)

`length` - value of Content-Length (<=0, no content)

`expires` - value of Expires (-1, no Expires header), (0, stop the refresh)

Returns: Returns the `OutputStream` to fill the content. If the update does not have new content (type = `null` and/or length = 0) method returns `null` and the message is sent automatically.

Throws:

`java.lang.IllegalArgumentException` - if some input parameter is invalid

javax.microedition.sip
SipRefreshListener

Declaration
```
public interface SipRefreshListener
```

Description
Listener interface for `RefreshHelper` events. This interface defines an event that contains a **`refreshID`** to identify the corresponding refresh task, **`statusCode`** that represent the result of this refresh (0 = cancelled, 200 = successful, else = failure). The **`statusCode`** corresponds to the response received for the original request sent by the `SipRefreshHelper`. The **`reasonPhrase`** gives a textual message about either success or failure of this refresh task.

See Also: `SipRefreshHelper`[48]

Member Summary	
Methods	
`void`	`refreshEvent(int refreshID, int statusCode, java.lang.String reasonPhrase)`[52]

Methods

refreshEvent(int, int, String)

Declaration:
```
public void refreshEvent(int refreshID, int statusCode, java.lang.String reasonPhrase)
```

Description:
Called when a refresh task is successfully started, refreshed, cancelled or failed.

Parameters:

`refreshID` - refresh ID

`statusCode` - the status code of the refresh task. If a response message was received the code corresponds to the response status code

`reasonPhrase` - additional textual message

javax.microedition.sip
SipException

Declaration

```
public class SipException extends java.io.IOException
```

```
java.lang.Object
  |
  +--java.lang.Throwable
        |
        +--java.lang.Exception
              |
              +--java.io.IOException
                    |
                    +--javax.microedition.sip.SipException
```

Description

This is an exception class for SIP specific errors. The exception includes free format textual error message and error code to categorize the error.

Member Summary

Fields

static byte	DIALOG_UNAVAILABLE[54]
static byte	GENERAL_ERROR[55]
static byte	INVALID_MESSAGE[55]
static byte	INVALID_OPERATION[54]
static byte	INVALID_STATE[54]
static byte	TRANSACTION_UNAVAILABLE[55]
static byte	TRANSPORT_NOT_SUPPORTED[54]
static byte	UNKNOWN_LENGTH[54]
static byte	UNKNOWN_TYPE[54]

Constructors

SipException(byte errorCode)[55]
SipException(java.lang.String message, byte errorCode)[55]

Methods

byte	getErrorCode()[56]

Inherited Member Summary

Methods inherited from class `Object`

`equals(Object), getClass(), hashCode(), notify(), notifyAll(), wait(), wait(), wait()`

Methods inherited from class `Throwable`

`getMessage(), printStackTrace(), toString()`

Fields

TRANSPORT_NOT_SUPPORTED

Declaration:
`public static final byte TRANSPORT_NOT_SUPPORTED`

Description:
The requested transport is not supported

See Also: SipConnection 9

DIALOG_UNAVAILABLE

Declaration:
`public static final byte DIALOG_UNAVAILABLE`

Description:
Thrown for example when SIP connection does not belong to any Dialog.

See Also: SipConnection.getDialog() 15

UNKNOWN_TYPE

Declaration:
`public static final byte UNKNOWN_TYPE`

Description:
Used when for example Content-Type is not set before filling the message body.

See Also: SipConnection.openContentOutputStream() 15

UNKNOWN_LENGTH

Declaration:
`public static final byte UNKNOWN_LENGTH`

Description:
Used when for example Content-Length is not set before filling the message body

See Also: SipConnection.openContentOutputStream() 15

INVALID_STATE

Declaration:
`public static final byte INVALID_STATE`

Description:
Method call not allowed, because of wrong state in SIP connection.

See Also: SipConnection 9, SipClientConnection 16, SipServerConnection 24

INVALID_OPERATION

Declaration:
`public static final byte INVALID_OPERATION`

Description:
The system does not allow particular operation. NOTICE! This error does not handle security exceptions.

See Also: SipConnection[9], SipClientConnection[16], SipServerConnection[24]

TRANSACTION_UNAVAILABLE

Declaration:
```
public static final byte TRANSACTION_UNAVAILABLE
```

Description:
System can not open any new transactions.

See Also: SipConnection[9], SipConnectionNotifier.acceptAndOpen()[29]

INVALID_MESSAGE

Declaration:
```
public static final byte INVALID_MESSAGE
```

Description:
The message to be sent has invalid format.

See Also: SipConnection.send()[12]

GENERAL_ERROR

Declaration:
```
public static final byte GENERAL_ERROR
```

Description:
Other SIP error

Constructors

SipException(byte)

Declaration:
```
public SipException(byte errorCode)
```

Description:
Construct SipException with error code.

Parameters:
`errorCode` - error code. If the error code is none of the specified codes the Exception is initialized with default GENERAL_ERROR.

SipException(String, byte)

Declaration:
```
public SipException(java.lang.String message, byte errorCode)
```

Description:
Construct SipException with textual message and error code.

Parameters:
`message` - error message.

`errorCode` - error code. If the error code is none of the specified codes the Exception is initialized with default GENERAL_ERROR.

Methods

getErrorCode()

Declaration:
```
public byte getErrorCode()
```

Description:
Gets the error code

CHAPTER 3

Annex A: Deploying JSR 180 on MIDP 2.0 Platform

A.1.0 Introduction

This section provides implementation notes for platform developers deploying the JSR 180 interfaces on a MIDP 2.0 platform. This section addresses features available in a MIDP 2.0 device that can be used to enhance SIP applications.

In particular, this document describes how to:

- use the MIDP 2.0 security features to control access to JSR180 capabilities
- use the MIDP 2.0 Push mechanism with SIP protocol and applications
- write applications to remain portable between the MIDP 1.0 and MIDP 2.0 platforms

A.2.0 Security

To send and receive messages (requests/responses) using this API, applications MUST be granted a permission to perform the requested operation. The mechanisms for granting a permission are implementation dependent.

A.2.1 Permissions for Opening Connections

The MIDP 2.0 specification defines a mechanism for granting permissions to use privileged features. This mechanism is based on a policy mechanism enforced in the platform implementation. The following permissions are defined for the JSR 180 functionality, when deployed with a JSR 118 MIDP 2.0 implementation.

To open a connection, a MIDlet suite requires an appropriate permission to access the SIPConnection implementation. If the permission is not granted, then `Connector.open` methods MUST throw a `SecurityException`. The following table indicates the permission that must be granted for each protocol.

Permission	Protocol
javax.microedition.io.Connector.sip	sip
javax.microedition.io.Connector.sips	sips

Note that ideally the permission granted will probably allow all subsequent network connections to be made (open a UDP/TCP/RTP media stream for example). However, this behavior might vary on different platforms.

A.3.0 JSR180 Push Capabilities and SIP Identity Sharing in MIDP 2.0

MIDP 2.0 defines a mechanism to register a MIDlet when a connection notification event is detected. Once the MIDlet has been launched it performs the same I/O operations it would normally use to open a connection and read and write data. For JSR 180 applications this capability allows the application to be launched if a message arrives either while the MIDlet is not running or while another MIDlet is running.

A.3.1 JSR 180 Push Registration Entry

Push registrations are either defined in the application descriptor or made dynamically at runtime via PushRegistry. The entry for a SIP protocol will include the connection URI string which identifies the scheme and port number of the inbound message connection. The entry also contains a filter field that designates which senders are permitted to send messages that launch the registered MIDlet. An asterisk (“*”) and question mark (“?”) can be used in the filter field as a wild cards as specified in the MIDP 2.0 specification.

For the SIP protocol, the filter field is matched against the URI of the From field in the received message.

It is also possible to add in the SIP URI a media feature tag (MIME type of the application) associated with the MIDlet. This tag is compared with the tag included in the received request:

- Media feature tag in Accept-Contact header field parameter:
  `Accept-Contact: *;type=“application/x-chess”`
- Or media type in the SDP field:
  `m=application 5062 SIP/TCP application/x-chess`

This functionality is specified in SIP Working Group (http://www.ietf.org/html.charters/sip-charter.html) as callee’s capabilities and caller’s preferences. The implementations must follow the framework and parameters specified in the final specification from WG. See also SIP identity sharing in this specification.

Push Registry entry examples:

- `MIDlet-Push-1: sip:5060, com.company.sip.SIPExample1, *@operator.com`
- `MIDlet-Push-2: sip:5080, com.company.sip.SIPExample2, *`
- `MIDlet-Push-3: sip:;type=“application/x-chess”, com.company.sip.SIPGame1, *`
- `MIDlet-Push-4: sip:5070;type=“application/x-cannons”, com.company.sip.SIPGame2, *`

Finally, the MIDlet can register dynamically to the Push Registry when opening the connection by passing the same augmented SIP URI to the Connector. Example:

```
String midletName = “com.company.sip.SIPExample2”;
String serverURI = “sip:5080”;
SipConnectionNotifier scn = (SipConnectionNotifier) Connector.open(sURI);
PushRegistry.registerConnection(serverURI, midletName, “*”);
```

Following table defines how the system is routing messages to MIDlets when registered in PushRegistry. It should be noticed that the system requires either port number or at least MIME type (case **2** shared port number) for the routing decision.

The definition of the previous functionality is intended to be common with JSR 205 (WMA 2.0) and will be updated accordingly.

	Connector SIP URI	Request routing and PushRegistry functionality
1	`sip:nnnn[;type="MIME type"]`	- example: `sip:5080;type="application/x-game"` - use dedicated port number `nnnn` - route incoming SIP request on port `nnnn` to this MIDlet and wake it up. If the optional MIME type feature tag is set, route only the requests with the matching tag.
2	`sip:;type="MIME type"`	- example: `sip:;type="application/x-game"` - share system SIP port (MIDlet does not have to know it) - route incoming SIP request with MIME type feature tag to this MIDlet and wake it up.
3	`sip:`	- use dedicated port selected by the system - route incoming SIP request to the selected port to this MIDlet - NOTE! URI `"sip:"` can not be registered in PushRegistry since both the port number and MIME type are unknown.

Unlike the initial push connections defined in MIDP 2.0, the SIP protocol includes an explicit buffering mechanism where messages are held until processed by some application that reads and deletes messages. If a message is delivered to the device and does not pass the specified filter, the message will be deleted by the Application Management Software.

When the application is started in response to a Push request, the application SHOULD read and process all messages that are buffered for it. If an application fails to read and process the messages when started or if starting of the application is denied (for example, by the end user), the platform implementation MAY delete unread messages from the buffer, if it becomes necessary to do so. For example, the platform implementation may delete messages when the buffer becomes full.

Another difference between the JSR180 interface and other protocol handlers in MIDP 2.0, is that JSR180 includes a SipConnectionNotifier which provides asynchronous callbacks when messages become available while the application is running.

A.4 Portable JSR180 applications between MIDP1.0 and MIDP2.0

If permitted by the device security policy, a JSR180 application written for a MIDP 1.0 platform will work without any modification on a MIDP 2.0 system. This behavior is defined by the MIDP 2.0 specification of untrusted applications. MIDP 2.0 also supports the concept of trusted applications. For these applications, the device can automatically handle trust decisions based on signed JAR files and a platform-specific policy mechanism that associates specific permissions with the signed application.

The security model also allows for the definition of user-granted permissions on a one-shot, session or blanket authorization. In many cases, the platform-dependent policy for permissions on MIDP 1.0 will be able to be mapped onto the MIDP 2.0 defined permissions. An application designed to work only on a MIDP 2.0 device can use the methods in the PushRegistry class to check if there are active connections (listConnections) or to add or remove registered connections at runtime (registerConnection or unregisterConnection).

An application designed to run portably on MIDP 1.0 or MIDP 2.0 platforms will only use the application descriptor and attributes in the manifest to describe requested permissions and push registration entries. See the MIDP 2.0 specification for details about the MIDlet-Permissions and MIDlet-Push-<n> attributes. On a MIDP

1.0 platforms these properties will be ignored. On a MIDP 2.0 platform, these properties will direct the application management software to perform the necessary checks and registrations when the application is installed and removed from the system.

CHAPTER 4

Annex B: Code examples

B.1.0 Establishing and stopping session - INVITE, 200 OK, ACK - BYE

Originating side

The MIDlet **(Listing B.3.1)** shows a simplified example of originating side UA in SIP session. The example handles only the SIP signaling in a sequence and does not handle any error cases. The MIDlet implements following things:

- Opens a `SipConnectionNotifier` for incoming requests like BYE and sets itself as a listener
- Opens outbound `SipClientConnection` to user *sippy.tester@example.com*
- Initializes and sends INVITE with attached SDP content
- Receives provisional responses 100, 180 until final 200 OK response
- Reads SDP content from 200 OK response
- Saves the `SipDialog` information after 200 OK response
- Initializes and sends ACK -> session established
- Gets new `SipClientConnection` from `SipDialog` object
- Initializes BYE request and sends it
- Receives 200 OK for BYE -> session terminated

Terminating side

This MIDlet **(Listing B.3.2)** shows an example of terminating side UA in SIP session. The example handles only the SIP signaling in a sequence and does not handle any error cases. The MIDlet does following steps:

- Opens inbound `SipConnectionNotifier` for incoming requests on “sip:5060”
- Receives INVITE request and reads the SDP content
- Initializes and sends responses 180 and 200 OK, with own SDP content
- Waits for other side to send ACK -> session established
- Waits for other side to send BYE -> session terminated

B.2.0 Subscribing for presence information

This MIDlet **(Listing B.3.3)** shows a simplified example of subscribing for presence info. The MIDlet does following things:

- Opens inbound `SipConnectionNotifier` for incoming requests on “sip:5060”
- Sets the MIDlet as a listener for events from `SipConnectionNotifier`

- Initializes and sends SUBSCRIBE with additional header information: Expires, Event and Accept
- Waits for 200 OK response
- Waits 10 seconds before sending un-SUBSCRIBE with “Expires: 0” header
- Receives 200 OK for un-SUBSCRIBE in `notifyResponse()` method
- The listener method `notifyResponse()` handles all NOTIFY requests

Listing B.3.1

```
import java.io.*;
import javax.microedition.io.*;
import javax.microedition.midlet.*;
import javax.microedition.lcdui.*;
import javax.microedition.sip.*;

public class OriginatingINVITE extends MIDlet {

    private Display display;
    private long startTime;
    private Form form;
    // using static SDP content as an example
    private String sdp = "v=0\n\no=bob 2890844730 2890844732 IN IP4 host.example.com\ns=
\nc=IN IP4 host.example.com\nt=0 0\nm=message 54344 SIP/TCP\na=user:bob";

    public OriginatingINVITE() {
        // Initialize MIDlet display
        display=Display.getDisplay(this);
        // create a Form for progess info printings
        form = new Form("Session example");
        display.setCurrent(form);
    }

    public void startApp() {
        SipConnectionNotifier scn = null;
        SipClientConnection scc = null;
        SipDialog dialog = null;
        try {
            // start a listener for possible incoming requests
            scn = (SipConnectionNotifier) Connector.open("sip:5060");
            // set this object as the listener
            scn.setListener(this);
            // open SIP connection with sippy.tester@10.128.0.50
            scc = (SipClientConnection)
                Connector.open("sip:sippy.tester@10.128.0.50:5070");
            // initialize INVITE request
            // and associate it with the SipConnectionNotifier
            scc.initRequest("INVITE", scn);
            scc.setHeader("Content-Length", ""+sdp.length());
            scc.setHeader("Content-Type", "application/sdp");
            OutputStream os = scc.openContentOutputStream();
            os.write(sdp.getBytes());
            os.close(); // close and send

            int statusCode = 0;
            boolean received = false;
            while(statusCode < 200) {
                received = scc.receive(15000); // blocking receive, with 15 sec timeout
                if(received) {
                    statusCode = scc.getStatusCode();
                    form.append("Response "+scc.getReasonPhrase());
                    if(statusCode == 200) {
                        String contentType = scc.getHeader("Content-Type");
                        String contentLength = scc.getHeader("Content-Length");
                        int length = Integer.parseInt(contentLength);
                        if(contentType.equals("application/sdp")) {
                            //
                            // handle SDP here
                            //
                        }
                        dialog = scc.getDialog(); // save dialog info
                    }
                } else {
                    form.append("Timeout no response...");
```

```
                }
            }
            scc.initAck(); // initialize and send ACK
            scc.send();
            scc.close();
            active = true;

            form.append("Wait 15 secs before closing the session...");
            synchronized(this) {
               try{ wait(15000); }catch(Exception ee ){ }
            }
            // if the session is still active
            if(active) {
                form.append("Closing session with BYE");
                // create SipClientConnection with
                // pre-initialized BYE from saved dialog
                scc = dialog.getNewClientConnection("BYE");
                scc.send();

                received = scc.receive(15000); // receive 200 OK
                if(received)
                    form.append("\nResponse: "+scc.getStatusCode()+" "+scc.getReasonPhra
se());
                active = false;
                scc.close();
            }
            destroyApp(true);
        } catch(IOException ex) {
            // handle IOException
        }
    }


    public void pauseApp() {
        // pause
    }

    public void destroyApp(boolean b) {
        notifyDestroyed();
    }

    public void shutdown() {
        destroyApp(false);
    }

    /**
     * listener method for incoming requests
     */
    public void notifyRequest(SipConnectionNotifier scn) {
        SipServerConnection ssc = null;
        try {
            ssc = scn.acceptAndOpen(); // fetch the new SipServerConnection
            if(ssc.getMethod().equals("BYE")) {
                // respond 200 OK to BYE
                ssc.initResponse(200);
                ssc.send();
                active = false;
                form.append("\nOther side hang-up!");
                form.append("\nClosing notifier...");
                ssc.close();
            } else {
                // 481 Call/Transaction Does Not Exist
                ssc.initResponse(481);
                ssc.send();
                ssc.close();
            }
        } catch(IOException ex) {
            // handle IOException
```

```
        }
    }

}
```

Listing B.3.2

```
import java.io.*;
import javax.microedition.io.*;
import javax.microedition.midlet.*;
import javax.microedition.lcdui.*;
import javax.microedition.sip.*;

public class TerminatingINVITE extends MIDlet {

    private Display display;
    private long startTime;
    private Form form;
    private boolean active = false;
    // using static SDP as an example
    private String sdp = "v=0\n\no=bob 2890844730 2890844732 IN IP4 host.example.com\ns=
\nc=IN IP4 host.example.com\nt=0 0\nm=message 54355 SIP/TCP\na=user:bob";

    public TerminatingINVITE() {
        // Initialize MIDlet display
        display = Display.getDisplay(this);
        form = new Form("Session example");
        display.setCurrent(form);
    }

    public void startApp() {
        SipServerConnection ssc = null;
        SipConnectionNotifier scn = null;

        try {
            // start a listener for incoming request
            scn = (SipConnectionNotifier) Connector.open("sip:5060");
            ssc = scn.acceptAndOpen(); // blocking
            if(ssc.getMethod().equals("INVITE")) {
                // handle content
                String contentType = ssc.getHeader("Content-Type");
                String contentLength = ssc.getHeader("Content-Length");
                int length = Integer.parseInt(contentLength);
                if(contentType.equals("application/sdp")) {
                    InputStream is = ssc.openContentInputStream();
                    byte content[] = new byte[length];
                    is.read(content);
                    String sc = new String(content);
                    // parse m= line from SDP, as an example
                    int m = sc.indexOf("m=");
                    String media = sc.substring(m, sc.indexOf('\n', m));
                    form.append("Otherside media is: "+media);
                    //
                    // handle media here
                    //

                    // initialize and send 180 response
                    ssc.initResponse(180);
                    ssc.send();
                    // inform user about the session here...
                }
                // accept automatically and initialize 200 response
                ssc.initResponse(200);
                ssc.setHeader("Content-Length", ""+sdp.length());
                ssc.setHeader("Content-Type", "application/sdp");
```

```
                OutputStream os = ssc.openContentOutputStream();
                os.write(sdp.getBytes());
                os.close(); // close and send
                ssc.close();

                // Wait for otherside to ACK
                form.append("Waiting for ACK...");
                ssc = scn.acceptAndOpen(); // blocking
                if(ssc.getMethod().equals("ACK")) {
                    form.append("Session established");
                } else {
                    // problems just stop
                    return;
                }

                // Wait for otherside to send BYE
                form.append("Waiting for BYE...");
                ssc = scn.acceptAndOpen(); // blocking
                if(ssc.getMethod().equals("BYE")) {
                    ssc.initResponse(200);
                    ssc.send();
                }
                ssc.close();
            }
            scn.close();
            destroyApp(true);
        } catch(IOException ex) {
            // handle IOException
        }
    }


    public void pauseApp() {
        // pause
    }

    public void destroyApp(boolean b) {
        notifyDestroyed();
    }

    public void shutdown() {
        destroyApp(false);
    }
}
```

Listing B.3.3

```
import java.io.*;
import javax.microedition.io.*;
import javax.microedition.midlet.*;
import javax.microedition.lcdui.*;

import javax.microedition.sip.*;

public class Subscribe extends MIDlet implements SipClientConnectionListener, SipServerC
onnectionListener {

    private Display display;
    private SipDialog dialog;
    private Form form;

    public Subscribe() {
        display=Display.getDisplay(this);
        form = new Form("SUBSCRIBE from server");
        display.setCurrent(form);
    }
```

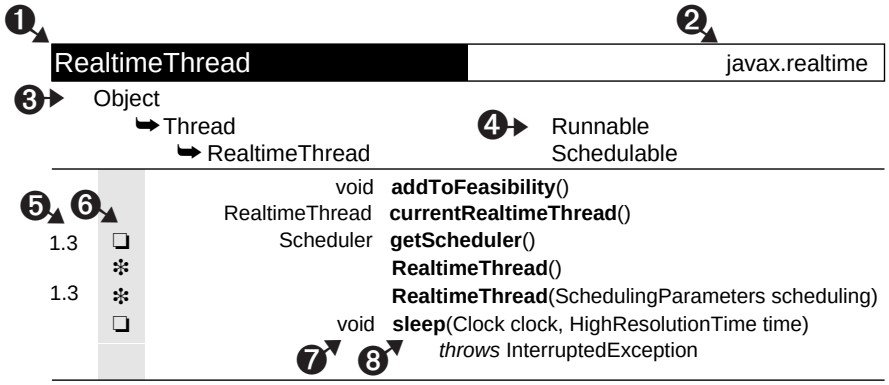
```
    public void startApp() {
        form.append("Starting...\n");
        try {
            SipConnectionNotifier scn = null;
            SipServerConnection ssc = null;
            // Start notifier on port 5060
            scn = (SipConnectionNotifier) Connector.open("sip:5060");
            scn.setListener(this);

            // Open outbound SIP connection to sippy.tester
            SipClientConnection scc =
               (SipClientConnection) Connector.open("sip:sippy.tester@sip.registrar.com"
);
            // Initialize and send SUBSCRIBE for 'presence' events
            scc.initRequest("SUBSCRIBE", scn);
            scc.setHeader("Expires", "930");
            scc.setHeader("Event", "presence");
            scc.addHeader("Accept", "application/xpidf+xml");
            scc.send();
            scc.receive(5000); // blocking receive
            form.append("Response: "+scc.getStatusCode());
            dialog = scc.getDialog();
            scc.close();

            form.append("Wait 10 secs before unsubscribing...");
            synchronized(this) {
               try{ wait(10000); }catch(Exception ee ){ }
            }
            // Initialize and send un-SUBSCRIBE, with Expires: 0
            scc = dialog.getNewClientConnection("SUBSCRIBE");
            // for example handle response in a listener
            scc.setListener(this);
            scc.setHeader("Expires", "0");
            scc.setHeader("Event", "presence");
            scc.send();
        } catch(IOException ex) {
            // handle IOException
        }
    }

    /**
     * listener method for incoming responses
     */
    public void notifyResponse(SipClientConnection scc) {
        try {
            scc.receive(0); // non-blocking receive
            form.append("Response: "+scc.getStatusCode());
            // handle response here
            scc.close();
        } catch(IOException ex) {
           // handle IOException
        }
    }

    /**
     * listener method for incoming requests
     */
    public void notifyRequest(SipConnectionNotifier scn) {
        SipServerConnection ssc;
        try {
            ssc = scn.acceptAndOpen(); // non-blocking
            form.append("Message: "+ssc.getMethod()+" received");
            // check if the received request is NOTIFY and it belongs to our dialog
            if(ssc.getMethod().equals("NOTIFY") && dialog.isSameDialog(ssc)) {
                String contentType = ssc.getHeader("Content-Type");
                String contentLength = ssc.getHeader("Content-Length");
                int length = Integer.parseInt(contentLength);
```

```
                if(contentType.equals("application/xpidf+xml")) {
                InputStream is = ssc.openContentInputStream();
                    byte buf[] = new byte[length];
                    int i = is.read(buf);
                    String tmp = new String(buf, 0, i);
                    form.append("NOTIFY info:"+tmp);
                    ssc.initResponse(200);
                    ssc.send();
                    //
                    // handle presence info
                    //
                }
            } else {
                // send 481 "Subscription does not exist"
                ssc.initResponse(481);
                ssc.send();
            }
            ssc.close();
        } catch(IOException ex) { }
    }

    public void pauseApp() {
    }

    public void destroyApp(boolean b) {
        notifyDestroyed();
    }

    public void shutdown() {
        destroyApp(false);
    }
}
```

ALMANAC LEGEND

The almanac presents classes and intefaces in alphabetic order, regardless of their package. Fields, methods and constructors are in alphabetic order in a single list.

This almanac is modeled after the style introduced by Patrick Chan in his excellent book *Java Developers Almanac*.



1. Name of the class, interface, nested class or nested interface. Interfaces are italic.
2. Name of the package containing the class or interface.
3. Inheritance hierarchy. In this example, `RealtimeThread` extends `Thread`, which extends `Object`.
4. Implemented interfaces. The interface is to the right of, and on the same line as, the class that implements it. In this example, `Thread` implements `Runnable`, and `RealtimeThread` implements `Schedulable`.
5. The first column above is for the value of the `@since` comment, which indicates the version in which the item was introduced.
6. The second column above is for the following icons. If the "protected" symbol does not appear, the member is public. (Private and package-private modifiers also have no symbols.) One symbol from each group can appear in this column.

Modifiers	**Access Modifiers**	**Constructors and Fields**
❍ abstract	♦ protected	❇ constructor
● final		✍ field
❑ static		
■ static final		

7. Return type of a method or declared type of a field. Blank for constructors.
8. Name of the constructor, field or method. Nested classes are listed in 1, not here.

CHAPTER 5

Almanac

SipAddress	**javax.microedition.sip**

Object
➥SipAddress

	String	**getDisplayName()**
	String	**getHost()**
	String	**getParameter(String name)**
	String[]	**getParameterNames()**
	int	**getPort()**
	String	**getScheme()**
	String	**getURI()**
	String	**getUser()**
	void	**removeParameter(String name)**
	void	**setDisplayName(String name)** ***throws*** **IllegalArgumentException**
	void	**setHost(String host)** ***throws*** **IllegalArgumentException**
	void	**setParameter(String name, String value)** ***throws*** **IllegalArgumentException**
	void	**setPort(int port)** ***throws*** **IllegalArgumentException**
	void	**setScheme(String scheme)** ***throws*** **IllegalArgumentException**
	void	**setURI(String URI)** ***throws*** **IllegalArgumentException**
	void	**setUser(String user)** ***throws*** **IllegalArgumentException**
❇		**SipAddress(String address)** ***throws*** **IllegalArgumentException**
❇		**SipAddress(String displayName, String URI)** ***throws*** **IllegalArgumentException**
	String	**toString()**

SipClientConnection	**javax.microedition.sip**

SipClientConnection — SipConnection

	int	**enableRefresh(SipRefreshListener srl)** ***throws*** **SipException**
	void	**initAck()** ***throws*** **SipException**
	SipClientConnection	**initCancel()** ***throws*** **SipException**
	void	**initRequest(String method, SipConnectionNotifier scn)** ***throws*** **IllegalArgumentException, SipException**
	boolean	**receive(long timeout)** ***throws*** **SipException, java.io.IOException**

void	**setCredentials(String username, String password, String realm)** ***throws*** **SipException**
void	**setListener(SipClientConnectionListener sccl)** ***throws*** **java.io.IOException**
void	**setRequestURI(String URI)** ***throws*** **IllegalArgumentException, SipException**

SipClientConnectionListener — javax.microedition.sip

SipClientConnectionListener

void	**notifyResponse(SipClientConnection scc)**

SipConnection — javax.microedition.sip

SipConnection — javax.microedition.io.Connection

void	**addHeader(String name, String value)** ***throws*** **SipException, IllegalArgumentException**
SipDialog	**getDialog()**
String	**getHeader(String name)**
String[]	**getHeaders(String name)**
String	**getMethod()**
String	**getReasonPhrase()**
String	**getRequestURI()**
int	**getStatusCode()**
java.io.InputStream	**openContentInputStream()** ***throws*** **java.io.IOException, SipException**
java.io.OutputStream	**openContentOutputStream()** ***throws*** **java.io.IOException, SipException**
void	**removeHeader(String name)** ***throws*** **SipException**
void	**send()** ***throws*** **java.io.IOException, java.io.InterruptedIOException, SipException**
void	**setHeader(String name, String value)** ***throws*** **SipException, IllegalArgumentException**

SipConnectionNotifier — javax.microedition.sip

SipConnectionNotifier — javax.microedition.io.Connection

SipServerConnection	**acceptAndOpen()** ***throws*** **java.io.IOException, java.io.InterruptedIOException, SipException**
String	**getLocalAddress()** ***throws*** **java.io.IOException**
int	**getLocalPort()** ***throws*** **java.io.IOException**
void	**setListener(SipServerConnectionListener sscl)** ***throws*** **java.io.IOException**

SipDialog — javax.microedition.sip

SipDialog

✍■	byte	**CONFIRMED**
✍■	byte	**EARLY**
	String	**getDialogID()**
	SipClientConnection	**getNewClientConnection(String method)** ***throws*** **IllegalArgumentException, SipException**
	byte	**getState()**
	boolean	**isSameDialog(SipConnection sc)**
✍■	byte	**TERMINATED**

SipException — javax.microedition.sip

Object
➥Throwable
➥Exception
➥java.io.IOException
➥SipException

✍■	byte	**DIALOG_UNAVAILABLE**
✍■	byte	**GENERAL_ERROR**
	byte	**getErrorCode()**
✍■	byte	**INVALID_MESSAGE**
✍■	byte	**INVALID_OPERATION**
✍■	byte	**INVALID_STATE**
❊		**SipException(byte errorCode)**
❊		**SipException(String message, byte errorCode)**
✍■	byte	**TRANSACTION_UNAVAILABLE**
✍■	byte	**TRANSPORT_NOT_SUPPORTED**
✍■	byte	**UNKNOWN_LENGTH**
✍■	byte	**UNKNOWN_TYPE**

SipHeader — javax.microedition.sip

Object
➥SipHeader

	String	**getHeaderValue()**
	String	**getName()**
	String	**getParameter(String name)**
	String[]	**getParameterNames()**
	String	**getValue()**
	void	**removeParameter(String name)**
	void	**setName(String name)** ***throws*** **IllegalArgumentException**
	void	**setParameter(String name, String value)** ***throws*** **IllegalArgumentException**
	void	**setValue(String value)** ***throws*** **IllegalArgumentException**
❊		**SipHeader(String name, String headerValue)** ***throws*** **IllegalArgumentException**
	String	**toString()**

SipRefreshHelper — javax.microedition.sip

Object
➥SipRefreshHelper

❑	SipRefreshHelper	**getInstance()**
	void	**stop(int refreshID)**
	java.io.OutputStream	**update(int refreshID, String contact, String type, int length, int expires)**

SipRefreshListener — javax.microedition.sip

SipRefreshListener

	void	**refreshEvent(int refreshID, int statusCode, String reasonPhrase)**

SipServerConnection	javax.microedition.sip
SipServerConnection	SipConnection
	void **initResponse(int code)** ***throws*** **IllegalArgumentException, SipException**
	void **setReasonPhrase(String phrase)** ***throws*** **SipException, IllegalArgumentException**

SipServerConnectionListener	javax.microedition.sip
SipServerConnectionListener	
	void **notifyRequest(SipConnectionNotifier ssc)**

CHAPTER 6

Constant Field Values

Contents

javax.microedition.*[7]

javax.microedition.sip.SipException	
static byte DIALOG_UNAVAILABLE[54]	2
static byte GENERAL_ERROR[55]	0
static byte INVALID_MESSAGE[55]	8
static byte INVALID_OPERATION[54]	6
static byte INVALID_STATE[54]	5
static byte TRANSACTION_UNAVAILABLE[55]	7
static byte TRANSPORT_NOT_SUPPORTED[54]	1
static byte UNKNOWN_LENGTH[54]	4
static byte UNKNOWN_TYPE[54]	3

Index

J

N

O

R

S

T

U